Proyecto Fin de Ciclo Superior

IES Hermanos Amorós

Creado por: Daniel Collado Albert

CICLO FORMATIVO DE GRADO SUPERIOR
DE DESARROLLO DE APLICACIONES WEB

Tabla de contenidos

1. Introducción....2
1.1 Descripción del proyecto....3
1.1.1 Situación Actual y Herramientas Existentes....3
1.1.2 Descripción General y Diferenciación....3
1.1.3 Público Objetivo....3
1.2 Objetivos....3
1.2.1 Interfaz Intuitiva y Facilidad de Uso....4
1.2.2 Panel de Control del Administrador....4
1.2.3 Sistema de Reseñas y Valoraciones....4
1.2.4 Seguridad....4
2.Analisis....4
2.1 Tecnologías empleadas....5
2.1.1 Piezas de software....5
2.1.1.1 Base de datos....5
2.1.1.2. Backend....5
2.1.1.2.1 Librerías Utilizadas en el backend....6
2.1.1.3 Frontend....11
2.1.1.3.1 Librerías Utilizadas en el frontend....12
2.1.2 Diagramas....13
2.1.2.1 Diagrama de Casos de Uso....14
2.1.3 Esquema de Base de Datos (Mongo DB)....17
3. Desarrollo....26
3.1 Estructura general del proyecto....27

3.1.1 Estructura del backend....27
3.1.2 Estructura del frontend....28
3.2 Gestión del proyecto....28
3.2.1 Fases del proyecto....28
3.2.2 Tareas y plazos de ejecución....28
3.2.3 Repositorio GIT....28
3.3 Bases de Datos....28
4. Funcionamiento de la Herramienta....28
4.1 Registro....28
4.2 Login....28
4.3 Tatuajes Realizados....28
4.3.1 Crear Tatuajes Realizados....28
4.3.2 Editar Tatuajes Realizados....28
4.3.3 Eliminar Tatuajes Realizados....28
4.4 Citas....28
4.4.1 Crear Citas....28
4.4.2 Editar Citas....28
4.4.3 Eliminar Citas....28
4.4.4 Ver Detalles Citas....28
4.5 Diseños Disponibles....28
4.5.1 Crear Diseños Disponibles....28
4.5.2 Editar Diseños Disponibles....28
4.5.3 Eliminar Diseños Disponibles....28
4.6 Reseñas....28
4.6.1 Crear Reseñas....28

4.6.2 Editar Reseñas.....28
Al guardar los cambios se nos notificara su correcta actualización con un mensaje de éxito:.....28
4.6.5 Estadísticas de Reseñas.....28
4.7 Lista de Usuarios.....28
4.7.1 Bloquear Usuario.....28
4.7.2 Buscar Usuario.....28
4.7.3 Cambiar Perfil.....28
4.7.4 Eliminar Usuarios.....28
5. Plan de Despliegue Local.....28
5.1 Preparación del Entorno de Desarrollo.....28
5.2 Clonación y Configuración del Proyecto.....28
5.3 Inicialización de la Base de Datos.....28
5.4. Iniciar la Aplicación en Modo de Desarrollo.....28
5.5 Desarrollo y Pruebas.....28
5.6 Validación Final.....28
5.7 Casuísticas y Recomendaciones Adicionales.....28
6. Conclusiones.....28
6.1 Logros Alcanzados.....28
6.2 Desafíos y Soluciones.....28
6.3 Futuras Mejoras.....28
7. Bibliografía.....28

1. Introducción

1.1 Descripción del proyecto

1.1.1 Situación Actual y Herramientas Existentes

Actualmente, la gestión de reservas en estudios de tatuajes se realiza mediante métodos tradicionales como llamadas o agendas físicas. Las plataformas genéricas de gestión de citas no satisfacen las necesidades específicas de estos estudios, lo que lleva a problemas como la doble reserva y la falta de seguimiento de clientes.

1.1.2 Descripción General y Diferenciación

Este proyecto desarrolla una página web para gestionar reservas en estudios de tatuajes, incluyendo frontend, backend y base de datos. La plataforma permitirá a los clientes ver piezas (tatuajes de mayor magnitud) del tatuador, así como diseños disponibles para poder reservar de manera eficiente la cita. Para los administradores, ofrecerá herramientas para gestionar reservas y administrar información usuarios y sus experiencias.

La plataforma se diferencia por su enfoque específico en tatuajes, incluyendo galería de portafolio, así como una página dedicada exclusivamente a los diferentes dibujos realizados por el tatuador para futuros tatuajes, con una interfaz intuitiva para mejorar la experiencia de usuarios y administradores.

1.1.3 Público Objetivo

La aplicación está dirigida a propietarios y administradores de estudios de tatuajes, artistas y clientes que buscan una solución eficiente para gestionar reservas y operaciones diarias.

1.2 Objetivos

1.2.1 Interfaz Intuitiva y Facilidad de Uso

Desarrollar una aplicación web con una interfaz intuitiva que permita a los usuarios crear citas de manera eficiente, ver portafolios del artista, poder escribir reseñas contando su experiencia y poder reservar citas.

1.2.2 Panel de Control del Administrador

El panel de control del administrador ofrecerá una visión completa de los usuarios registrados en el sistema, desde el cual podrá eliminar a un usuario o si lo prefiere bloquearlo, dejando un mensaje del motivo del bloqueo, además, de poder cambiar de rol a los diferentes usuarios por si quiere promocionar o degradar a algún usuario.

1.2.3 Sistema de Reseñas y Valoraciones

Se integrará un sistema de reseñas para permitir a los clientes dejar feedback sobre sus experiencias, ayudando a otros clientes a tomar decisiones informadas y proporcionando retroalimentación valiosa para mejorar los servicios.

1.2.4 Seguridad

La seguridad será una prioridad fundamental en nuestro sistema. Implementaremos medidas robustas y avanzadas para garantizar la protección integral de los datos de los usuarios. Además, proporcionaremos orientación y soporte para guiarlos adecuadamente en caso de que cometan algún error, asegurando así una experiencia segura y confiable.

2.Analisis

2.1 Tecnologías empleadas

2.1.1 Piezas de software

2.1.1.1 Base de datos

Para la gestión de la base de datos, se ha seleccionado MongoDB debido a las múltiples ventajas que ofrece en comparación con otras soluciones. Entre sus principales beneficios destacan:

- **Modelo de datos flexible:** Al ser una base de datos NoSQL, MongoDB almacena la información en documentos JSON lo que permite una estructura dinámica.

- **Alta escalabilidad:** Su arquitectura facilita la escalabilidad horizontal mediante la distribución de datos.

- **Alto rendimiento:** MongoDB optimiza el acceso a los datos a través de índices y almacenamiento en memoria, lo que mejora la velocidad de lectura y escritura en comparación con bases de datos relacionales tradicionales.

- **Facilidad de integración:** Gracias a su compatibilidad con diversos lenguajes de programación y frameworks, facilita la implementación en entornos modernos de desarrollo web.
  Por estas características, MongoDB se eligió como base de datos para este proyecto, garantizando una gestión eficiente y adaptable a los requerimientos del sistema.



2.1.1.2. Backend

Para el desarrollo del backend, se ha seleccionado Node.js junto con Express.js debido a sus ventajas en términos de rendimiento, escalabilidad y facilidad de implementación.

Las principales razones para esta elección son:

- **Eficiencia y rendimiento:** Node.js permite ejecutar código de manera rápida y optimizada. Su modelo asíncrono mejora el manejo de múltiples solicitudes.

- **Escalabilidad:** Gracias a su arquitectura basada en eventos, permite desarrollar aplicaciones que gestionan un alto volumen de tráfico de manera eficiente.

- **Desarrollo rápido y flexible:** Express.js proporciona una estructura ligera y sencilla para gestionar rutas, middleware y controladores con facilidad.

- **Amplia comunidad y soporte:** Node.js cuenta con una gran cantidad de paquetes disponibles en npm, facilitando la integración de nuevas funcionalidades.

- **Compatibilidad con MongoDB:** Ambos utilizan JSON para el intercambio de datos, simplificando la comunicación entre el backend y la base de datos.

Esta combinación ha sido elegida para garantizar un desarrollo eficiente y un backend optimizado.



2.1.1.2.1 Librerías Utilizadas en el backend

1. **Node.js:**

   a. **Descripción:** Node.js es un entorno de ejecución de JavaScript en el servidor que permite construir aplicaciones escalables y de alto rendimiento.

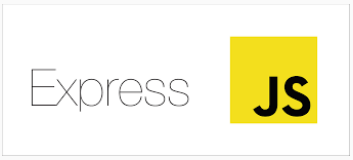
   b. Uso: Se utiliza para desarrollar aplicaciones del lado del servidor, especialmente aquellas que requieren manejo de operaciones de entrada/salida intensivas.



2. **Express**:

   a. **Descripción:** Express es un framework minimalista para Node.js que facilita la creación de APIs y el manejo de rutas.

   b. **Uso:** Se utiliza para construir rápidamente aplicaciones web y APIs RESTful en Node.js.



3. **Mongoose:**

   a. **Descripción:** Mongoose es una librería para Node.js que facilita la interacción con MongoDB, No es una base de datos en sí misma, sino una herramienta que ayuda a estructurar los datos y realizar operaciones en MongoDB de manera más organizada y eficiente.

   b. **Uso:** Se utiliza para definir esquemas y modelos de datos, realizar consultas y actualizaciones en MongoDB, y gestionar relaciones entre diferentes colecciones de datos.



4. **JWT (JSON Web Token):**

   a. **Descripción:** JWT es un estándar para la creación de tokens de acceso que permiten la autenticación segura entre partes.

   b. **Uso:** Se utiliza para implementar sistemas de autenticación seguros en aplicaciones web.



5. **CORS (Cross-Origin Resource Sharing):**

   a. **Descripción:** CORS es un módulo que permite manejar solicitudes de recursos desde diferentes dominios.

   b. **Uso:** Se utiliza para habilitar CORS en las rutas de una aplicación Express, permitiendo interacciones seguras entre diferentes dominios.

CORS
Cross-Origin Resource Sharing

6. **Log4js:**

   a. **Descripción:** Log4js es una librería de registro (logging) para Node.js que permite registrar mensajes en diferentes niveles de severidad.

   b. Uso: Se utiliza para gestionar y registrar eventos y errores en la aplicación, facilitando el monitoreo y la depuración.

7. **Morgan:**

   a. **Descripción:** Morgan es un middleware de registro de solicitudes HTTP para Node.js.

   b. **Uso:** Se utiliza para registrar detalles de las solicitudes HTTP en aplicaciones Express, ayudando en la depuración y el monitoreo.



8. **Multer:**

   a. **Descripción:** Multer es un middleware para manejar datos de formularios **multipart/form-data**, utilizado principalmente para la carga de archivos.

   b. **Uso:** Se utiliza para manejar la subida de archivos en aplicaciones web, como imágenes o documentos.



9. **Bcrypt:**

    a. **Descripción:** Bcrypt es una librería para el hashing de contraseñas, proporcionando una manera segura de almacenar contraseñas.

    b. **Uso:** Se utiliza para cifrar contraseñas antes de almacenarlas en la base de datos, mejorando la seguridad de la aplicación.



2.1.1.3 Frontend

Para el desarrollo del frontend, se ha elegido React debido a sus múltiples ventajas en términos de eficiencia, modularidad y experiencia de usuario.

Las principales razones para esta elección son:

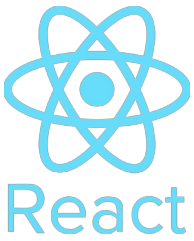
- **Componentización:** React permite dividir la interfaz en componentes reutilizables, facilitando el mantenimiento y la escalabilidad del proyecto.

- **Alto rendimiento:** Gracias al Virtual DOM, optimiza la actualización de la interfaz, reduciendo el tiempo de renderizado y mejorando la velocidad de la aplicación.

- **React Hooks:** Facilita la gestión del estado de la aplicación de manera eficiente sin necesidad de librerías adicionales.

- **Integración con backend:** Permite una comunicación fluida con APIs mediante fetch o Axios, optimizando la gestión de datos desde el servidor.

Debido a estas ventajas, React ha sido seleccionado como la tecnología principal para la construcción del frontend.



2.1.1.3.1 Librerías Utilizadas en el frontend

1. **SweetAlert**:

   a. **Descripción:** SweetAlert es una librería para crear alertas y diálogos personalizados y atractivos en aplicaciones web.

   b. Uso: Se utiliza para reemplazar las alertas predeterminadas del navegador con diálogos más estéticos y personalizables, mejorando la experiencia de usuario. Es ideal para mostrar mensajes de éxito, error, confirmación, etc., de una manera más amigable y visualmente atractiva.



2. **Axios:**

   a. **Descripción:** Axios es una librería para realizar solicitudes HTTP desde el navegador o Node.js.

   b. **Uso:** Se utiliza para manejar solicitudes y respuestas HTTP de manera sencilla y eficiente. Axios es muy útil para interactuar con APIs, permitiendo enviar y recibir datos de manera asíncrona. Es especialmente útil para operaciones CRUD (Crear, Leer, Actualizar, Eliminar) en aplicaciones web.

AXIOS

3. **UUID (Universally Unique Identifier)**

   a. **Descripción:** UUID es un identificador único de 128 bits utilizado para identificar información en sistemas informáticos.

   b. **Uso:** Utilizado en bases de datos para identificar de manera única elementos que requieran de una selección especifica.

{uuid}

2.1.2 Diagramas

Para una mejor comprensión y organización de las diferentes acciones que pueden realizar los distintos tipos de usuarios que interactúan con la página web, se han desarrollado varios diagramas.

2.1.2.1 Diagrama de Casos de Uso

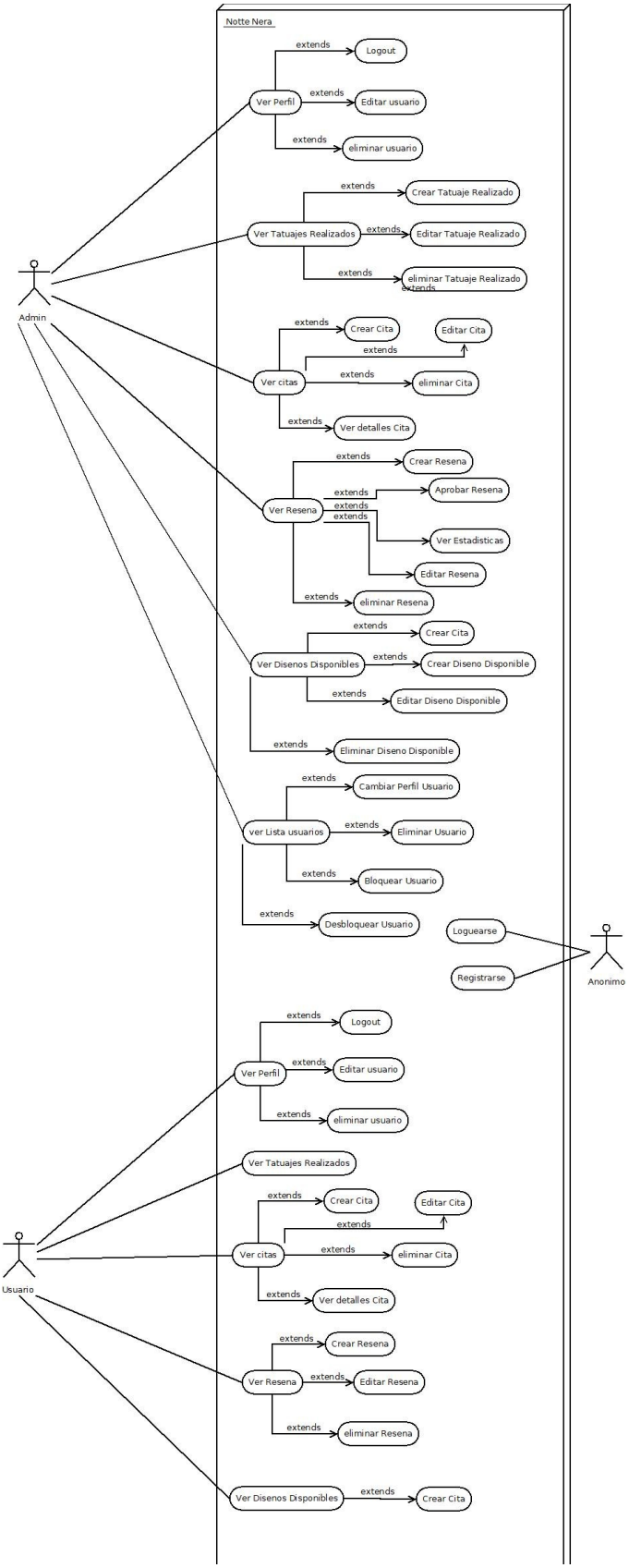
El siguiente diagrama representa todas las acciones contempladas para cada tipo de usuario que interactúa con la plataforma. En este caso, se han definido tres perfiles: usuario anónimo, usuario registrado y administrador.

- **Usuario anónimo:** Solo puede registrarse o iniciar sesión.

- **Usuario registrado:** Tiene acceso a la mayoría de las funcionalidades del sitio, pudiendo visualizar contenido, gestionar citas y dejar reseñas.

- **Administrador:** Además de contar con los mismos permisos que el usuario registrado, tiene la capacidad de gestionar el contenido de la plataforma. Puede aprobar o eliminar reseñas, actualizar diseños disponibles y tatuajes realizados, así como bloquear o eliminar usuarios cuando sea necesario.

Este diagrama permite visualizar de manera clara los permisos y responsabilidades de cada perfil dentro del sistema.

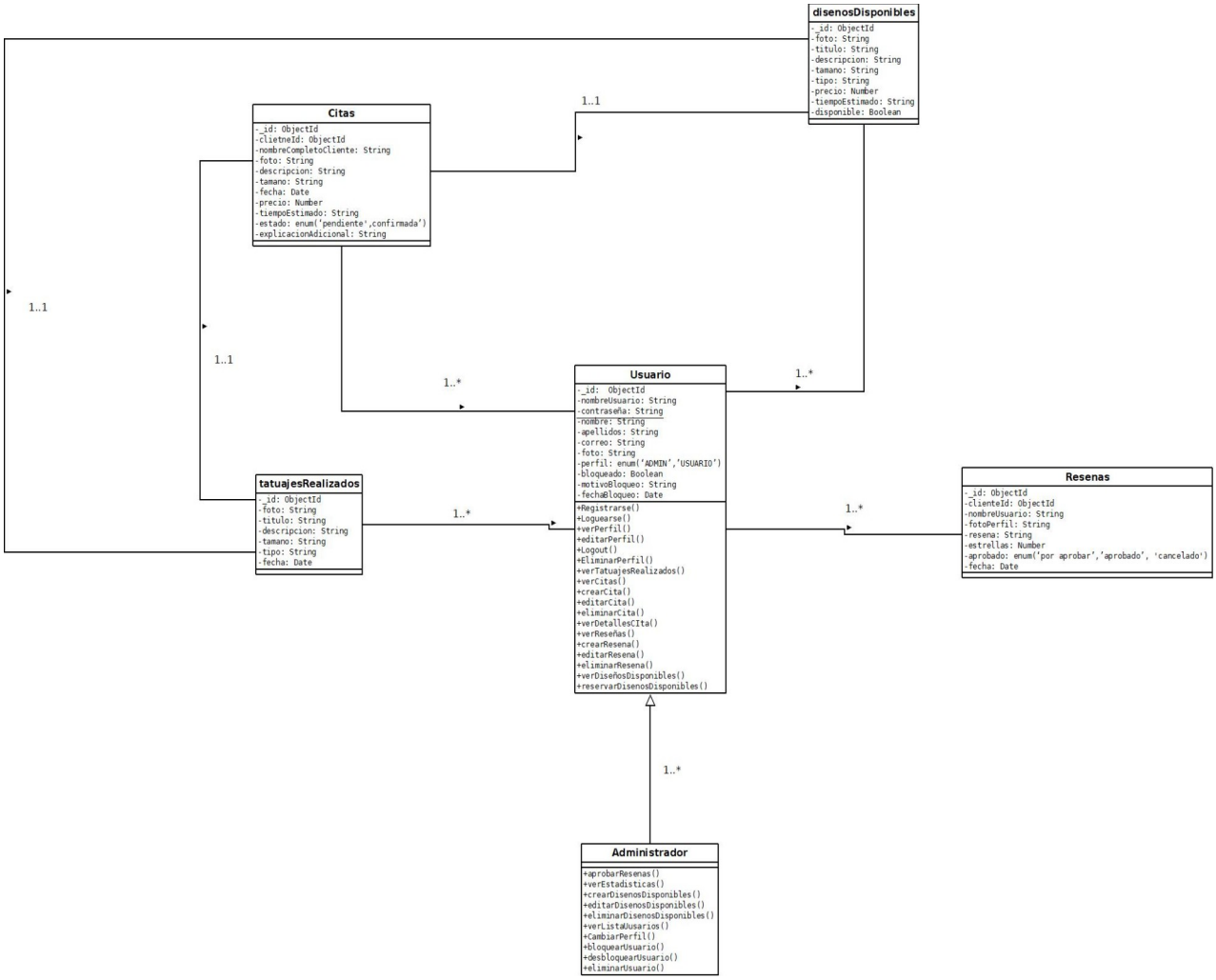

Notte Nera
Admin
Usuario
Anonimo
Logout
Ver Perfil
Editar usuario
eliminar usuario
Crear Tatuaje Realizado
Ver Tatuajes Realizados
Editar Tatuaje Realizado
eliminar Tatuaje Realizado
Crear Cita
Editar Cita
Ver citas
eliminar Cita
Ver detalles Cita
Crear Resena
Aprobar Resena
Ver Resena
Ver Estadisticas
Editar Resena
eliminar Resena
Crear Cita
Ver Disenos Disponibles
Crear Diseno Disponible
Editar Diseno Disponible
Eliminar Diseno Disponible
Cambiar Perfil Usuario
ver Lista usuarios
Eliminar Usuario
Bloquear Usuario
Desbloquear Usuario
Loguearse
Registrarse
extends

2.1.2.2 Diagrama de Clases

En el siguiente diagrama se representan las clases del sistema, detallando los atributos que requiere cada modelo y las acciones que pueden realizarse sobre ellos.

Además, se diferencia entre las acciones disponibles para un usuario registrado y aquellas exclusivas del administrador. Mientras que el usuario puede gestionar sus citas, dejar reseñas y visualizar tatuajes disponibles, el administrador tiene permisos adicionales, como la aprobación o eliminación de reseñas, la administración de citas y la gestión de usuarios.

Este diagrama es clave para comprender la estructura del backend y la interacción entre los distintos elementos del sistema.



2.1.3 Esquema de Base de Datos (Mongo DB)

A continuación, definimos el esquema de base de datos, estructura lógica que define cómo se organizan y relacionan los datos dentro de la base de datos. Actúa como un plano que describe las tablas, campos, relaciones y restricciones que la conforman. El esquema ofrece una visión general de la estructura de los datos sin incluir los datos reales.

Diagrama base:

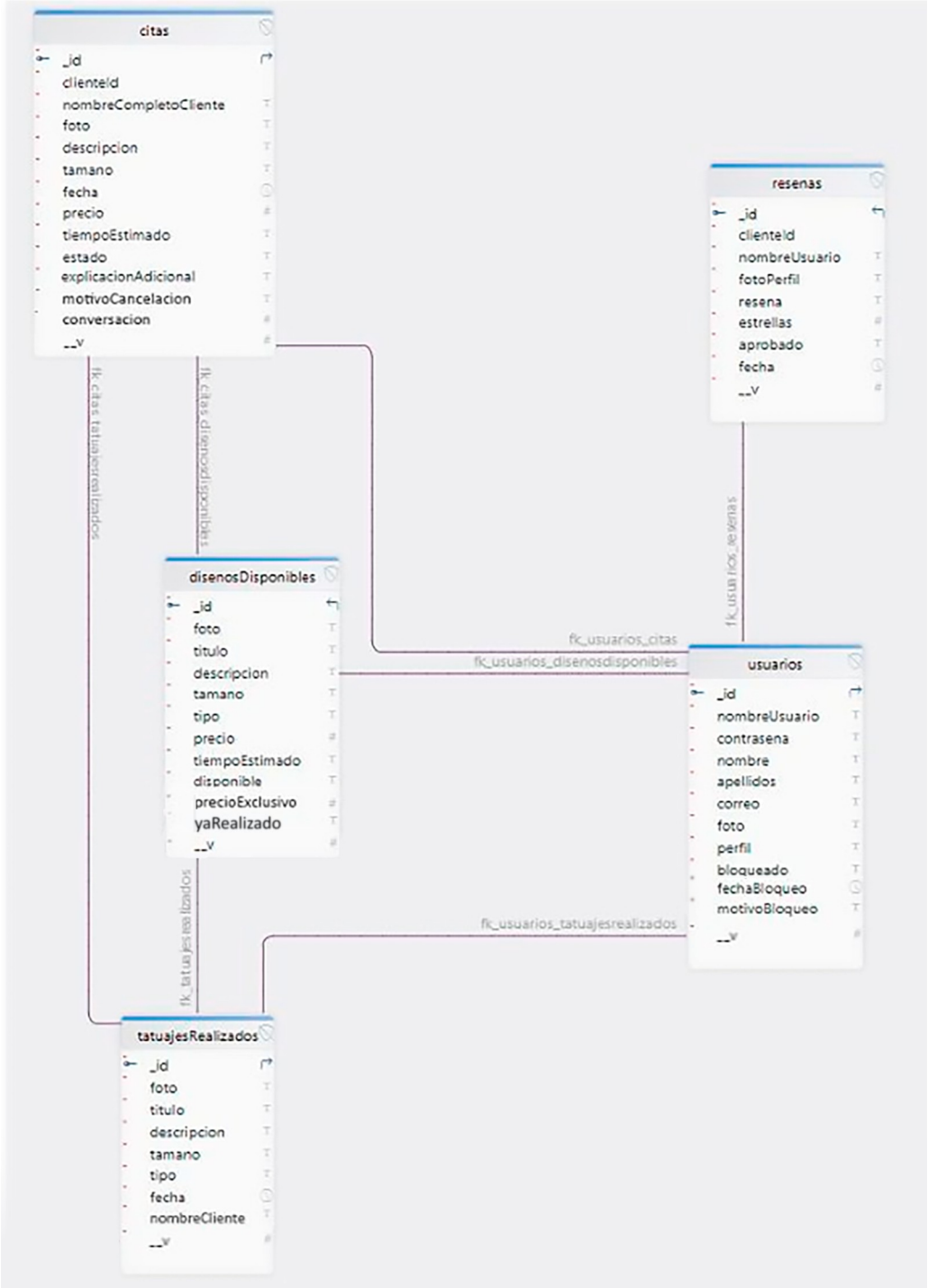


Tabla de usuarios:

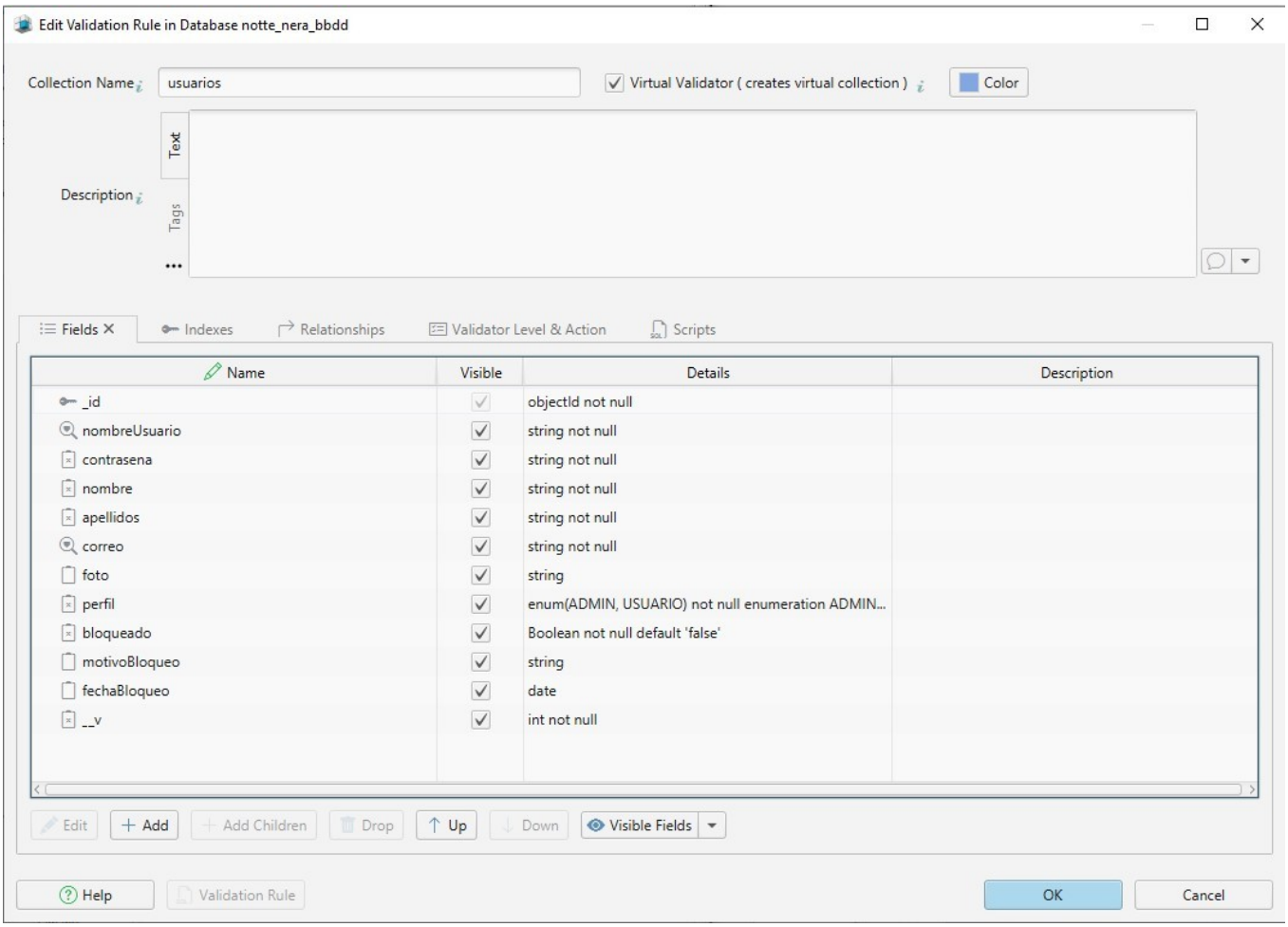


Tabla tatuajes realizados:

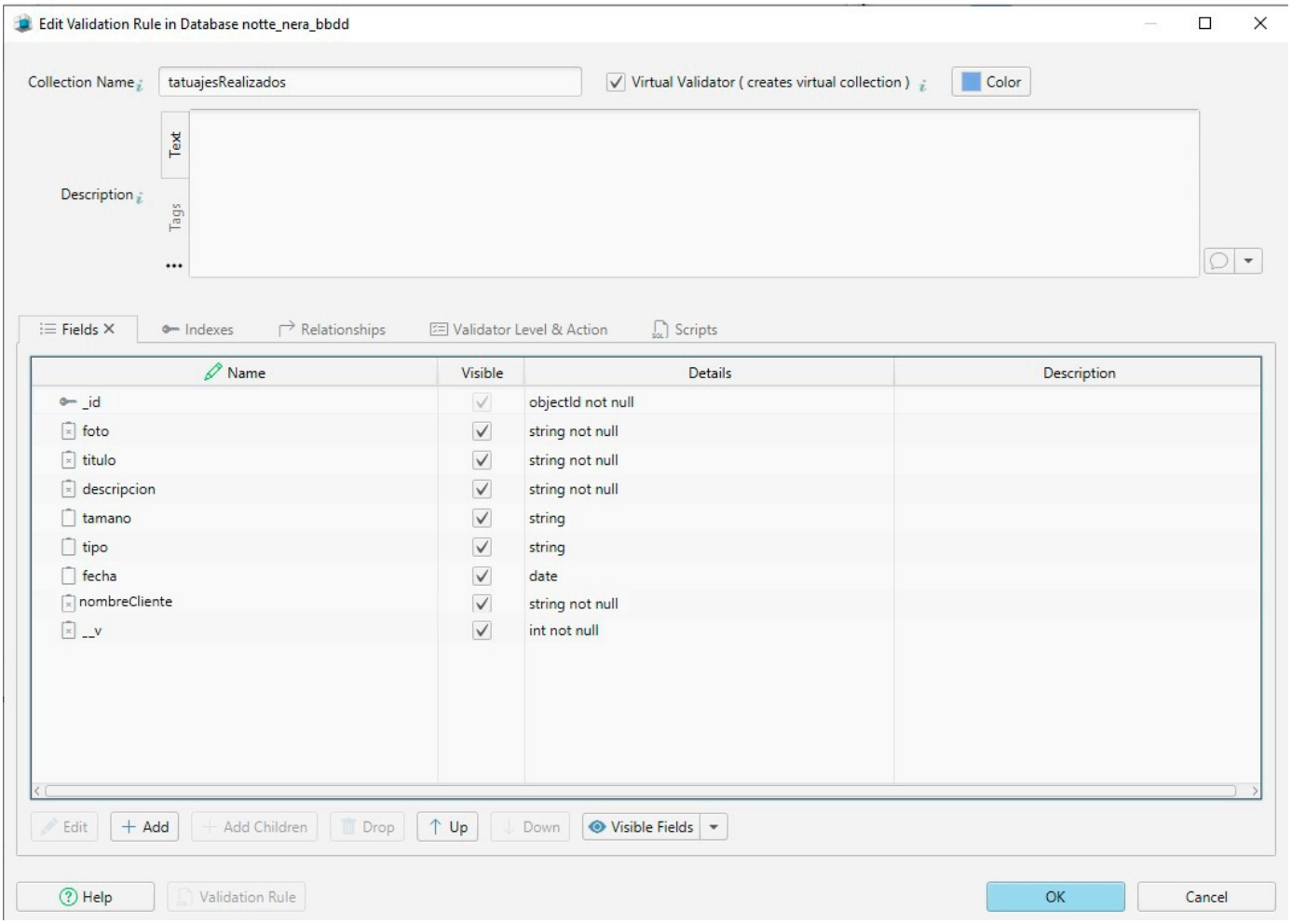


Tabla de citas:

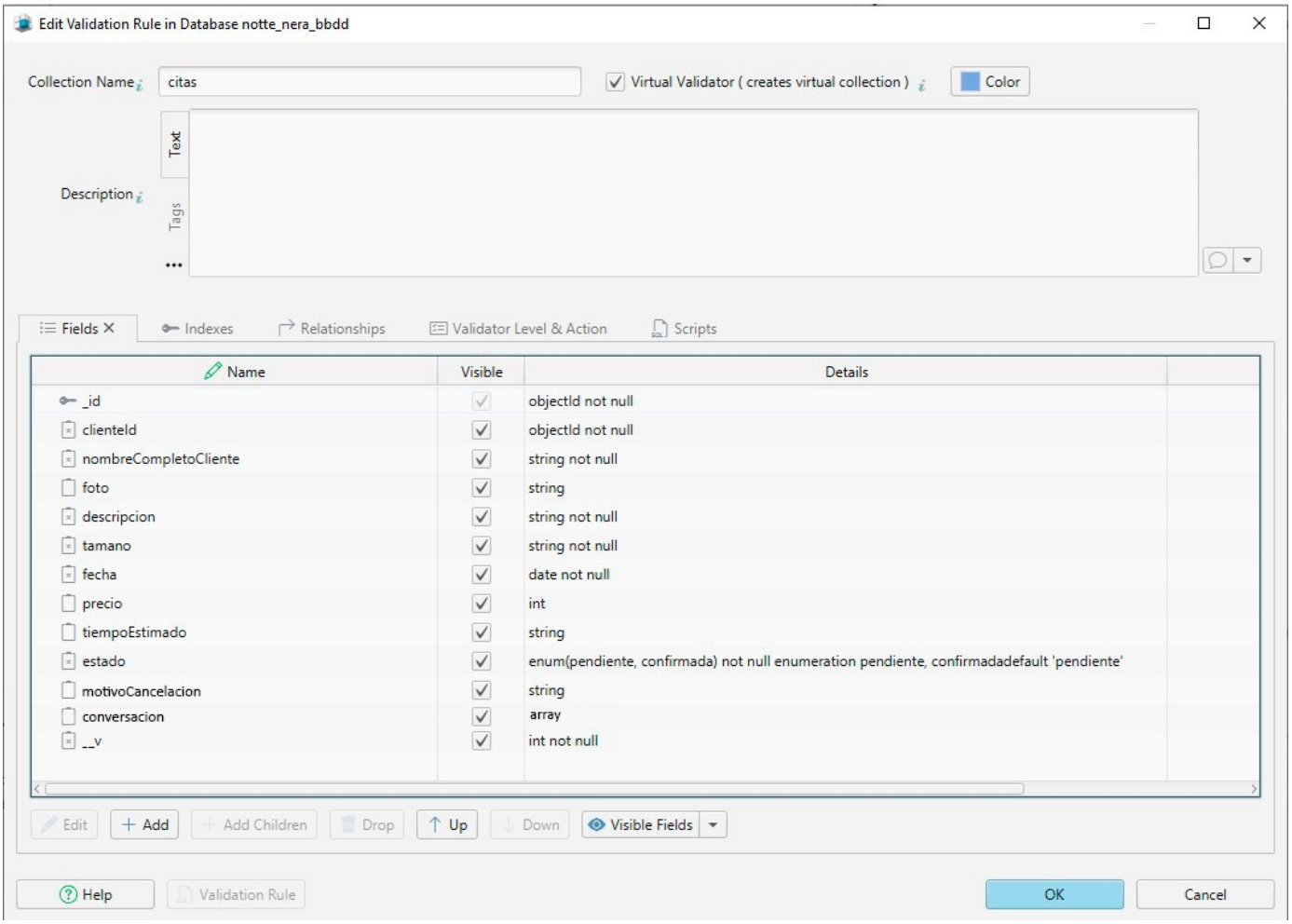


Tabla de diseños disponibles:

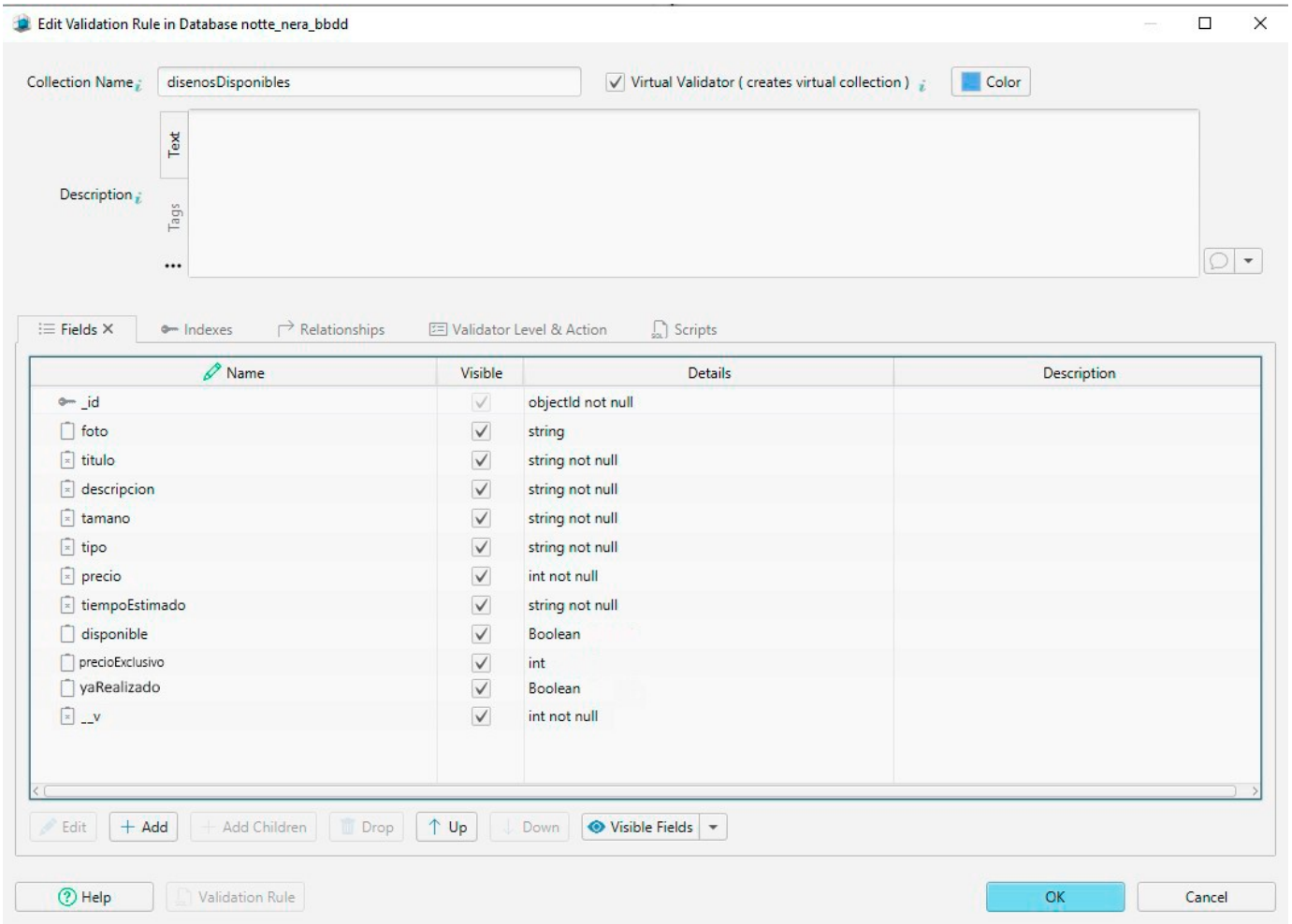
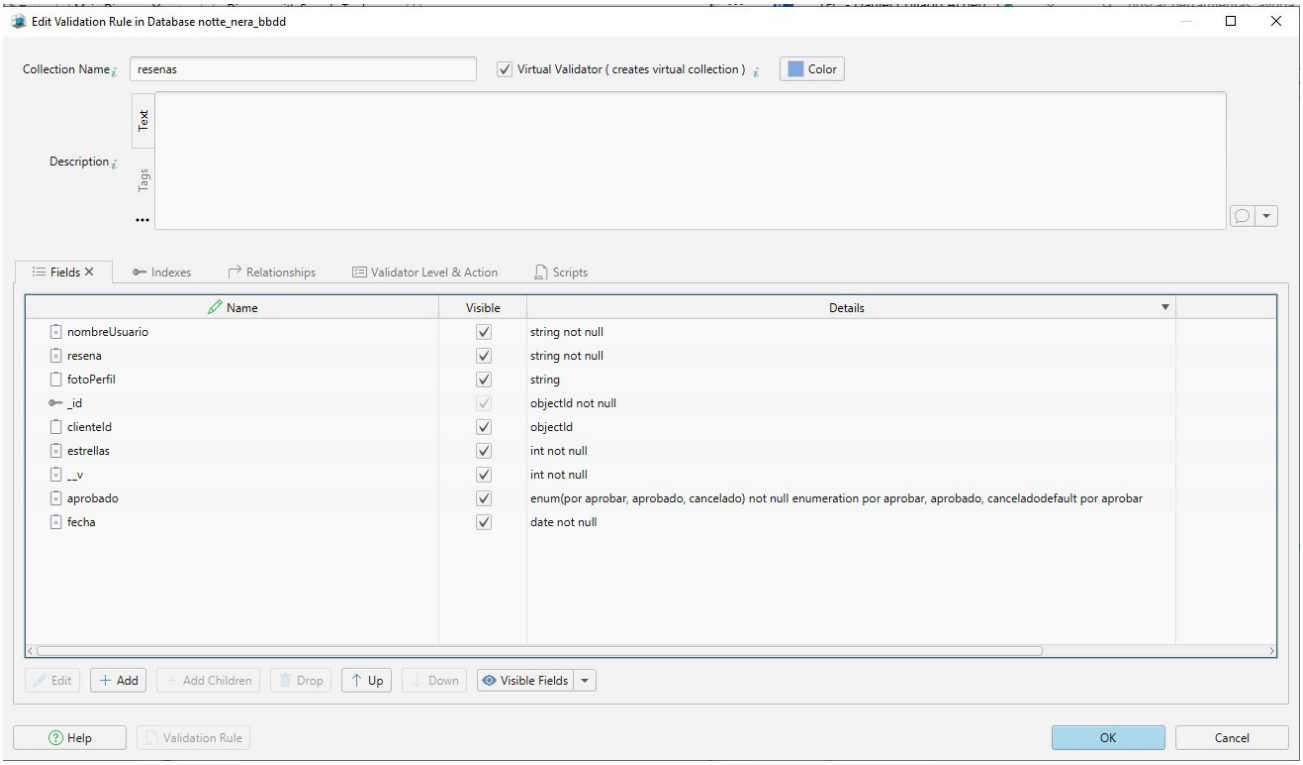


Tabla de reseñas:



2.2 Recursos materiales y personales

Además de las tecnologías principales mencionadas anteriormente, se han utilizado diversos programas y herramientas complementarias para apoyar el desarrollo del proyecto:

- **Photoshop:** Software utilizado para realizar el mockup de las distintas páginas del proyecto, proporcionando una referencia visual que será implementada posteriormente en la fase de desarrollo.

- **Word:** Herramienta utilizada para la elaboración de la documentación del proyecto.

- **Dia:** Programa utilizado para la creación de diagramas de casos de uso y de clases

- **Trello:** Herramienta de gestión de proyectos empleada para organizar las tareas del trabajo, facilitando el seguimiento del progreso en las distintas fases del proyecto.

- **Visual Studio Code:** Entorno de desarrollo integrado (IDE) utilizado para la programación, debido a su amplia gama de extensiones y características que mejoran la productividad y facilitan el desarrollo de aplicaciones.

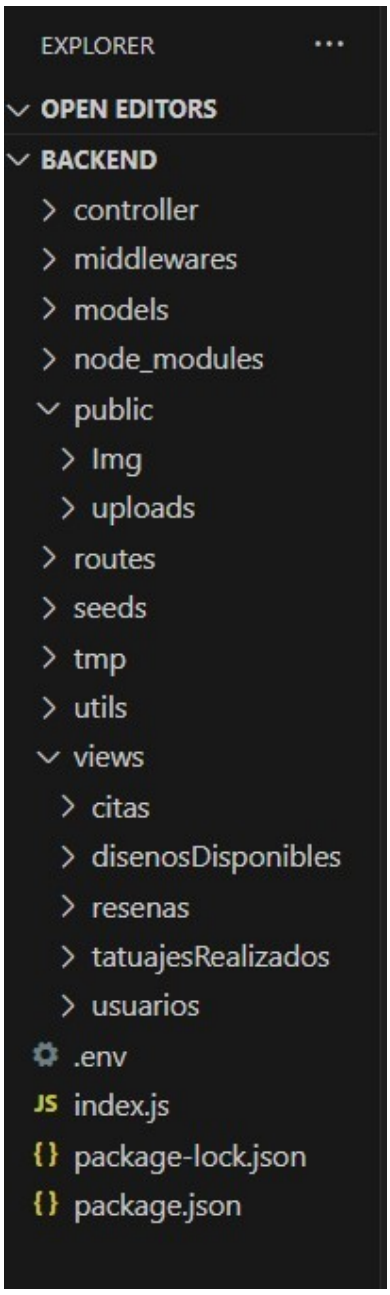
- **GitHub:** Plataforma utilizada para el control de versiones del proyecto.

- **Insomnia:** Programa empleado para probar el backend, debido a su utilización en la creación y envío de solicitudes HTTP, permitiendo verificar el correcto funcionamiento de la API desarrollada.

- **DbSchema:** Software utilizado para la creación y gestión de esquemas de bases de datos, permitiendo una visualización clara de las relaciones y estructuras de las tablas.

3. Desarrollo

3.1 Estructura general del proyecto



3.1.1 Estructura del backend

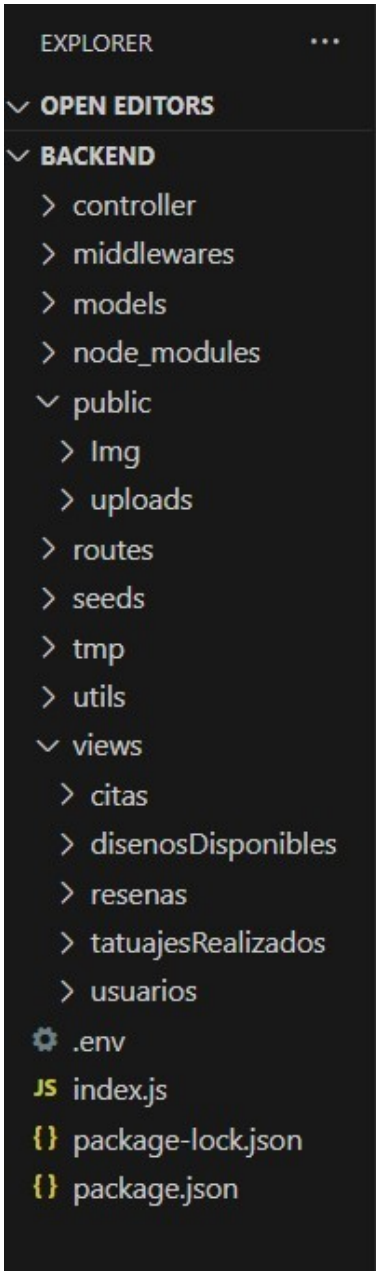
Como se puede observar en la imagen, esta se trata del sistema de encarpetado del backend, que consistirá en los siguientes apartados:

- **Controller:** Actúa como intermediario entre el modelo y la vista. Su función es recibir las entradas del usuario, procesar los datos a través del modelo y actualizar la vista según sea necesario, gestionando así la lógica de la aplicación.

- **Middleware:** En esta carpeta se almacenarán los fragmentos de código que deben ejecutarse de forma constante, ya sea por razones de seguridad o para registrar posibles errores.

- **Models:** Contendrá las definiciones de la estructura de los datos, la relación entre ellos y cómo deben manipularse dentro del sistema.

- **Node_modules:** Carpeta en la que se almacenan los módulos instalados necesarios para el correcto funcionamiento de la aplicación.



- **Public:** Directorio utilizado para almacenar archivos estáticos que se sirven directamente a los clientes sin necesidad de procesamiento adicional por parte del servidor.

  - **Uploads:** Se almacenarán las fotos subidas tanto por los usuarios como por los administradores, tales como imágenes de perfiles o documentos asociados a la plataforma.

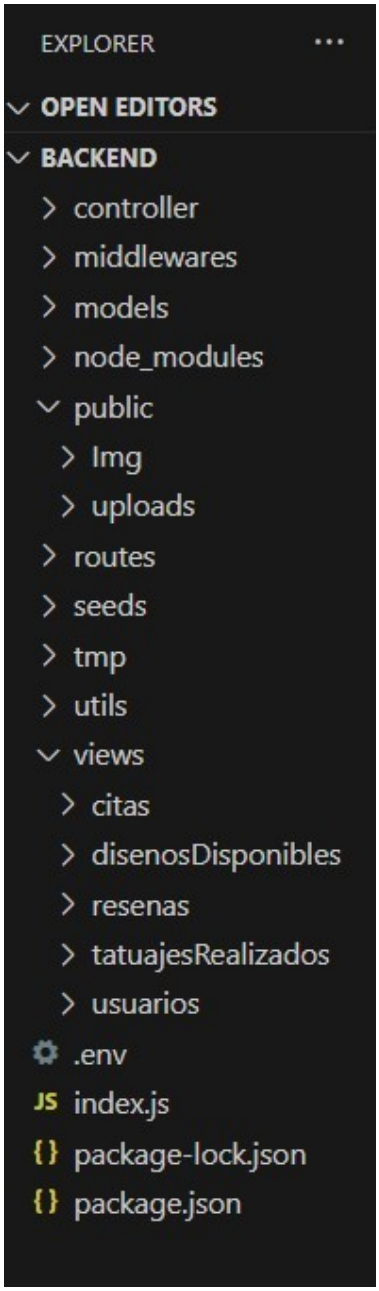
- **Routes:** Carpeta que contiene las rutas de las diferentes páginas o secciones de la aplicación.

- **Seeds:** Contendrá pequeños scripts que permiten rellenar rápidamente las tablas de la base de datos para evitar que estas estén vacías durante el desarrollo.

- **Tmp:** Archivos temporales que servirán como registro de las actividades del sistema, como los accesos a la web y los diferentes errores que puedan presentarse durante la navegación.

- **Utils:** Carpeta destinada a almacenar código auxiliar que centraliza tareas comunes o de utilidad, facilitando la organización y el mantenimiento del proyecto.

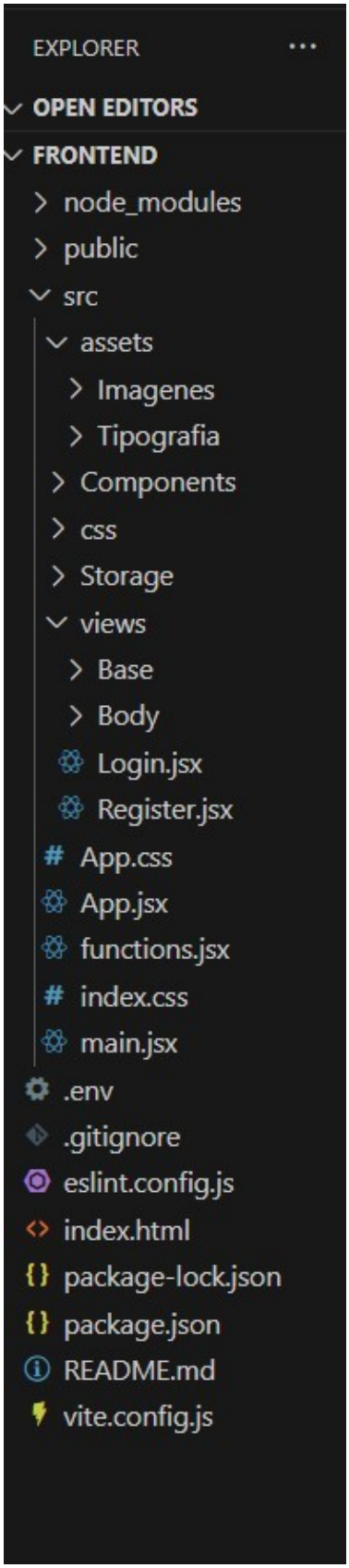
- **Views:** Carpeta que se utilizará durante el desarrollo del backend, mediante renderizado del lado del servidor (server-side rendering), para permitir la interacción con la página mientras se encuentra en construcción.



- **.env:** Archivo utilizado para almacenar variables de entorno del proyecto, que incluyen configuraciones sensibles, como claves API, credenciales de bases de datos o configuraciones de puertos. Estas variables no deben estar directamente en el código fuente.

- **Index.js:** Archivo de configuración del servidor, que actúa como la base del servidor de la aplicación.

- **Package.json:** Archivo en formato JSON que recoge todos los módulos y dependencias instalados en el proyecto.



3.1.2 Estructura del frontend

Ahora comenzaremos con la explicación del encarpetado por parte del frontend

- **Node_modules:** Carpeta que contiene todas las dependencias y paquetes instalados para el proyecto.

- **Src:** Directorio principal donde se encuentra el código fuente del proyecto. Aquí se desarrollan los componentes, estilos y funciones de la aplicación.

- **Assets:** Contiene recursos estáticos como imágenes, fuentes y otros archivos que pueden ser utilizados en la interfaz del usuario.

  - **Imágenes:** Carpeta destinada a almacenar las imágenes recurrentes del sistema, como la imagen de la estrella de reseñas o la foto por defecto de usuario.

  - **Tipografía:** Contiene archivos de la tipografía de la página web para favorecer el diseño.

- **Components:** Directorio donde se almacenan los diferentes componentes reutilizables de React.

- **Css:** Carpeta donde se encuentran los archivos de hojas de estilo para personalizar la apariencia de la aplicación.

- **Storage:** Carpeta donde se haya el código de Storage.jsx, que proporciona métodos para

3.2 Gestión del proyecto

3.2.1 Fases del proyecto



Las distintas fases que conforman este proyecto son las siguientes:

- **Planificación:** En esta fase se definirán las diferentes vistas de la página, su estética y los permisos asignados a cada tipo de usuario: administrador, usuario y visitante anónimo.

- **Documentación:** A medida que avance el desarrollo, se documentará cada parte del proyecto para garantizar que la información se mantenga actualizada y evitar posibles imprecisiones en el futuro.

- **Desarrollo del Backend:** Lo primero a desarrollar sería el servidor y toda la funcionalidad de este.

- **Depuración backend:** Una vez finalizado el desarrollo del backend, se realizarán pruebas exhaustivas de cada funcionalidad para detectar y corregir comportamientos inesperados o errores.



- **Desarrollo del Frontend:** Una vez finalizado el backend, se procederá con el desarrollo del frontend, optimizando la apariencia de la página y asegurando una experiencia visual atractiva y cómoda para el usuario.

- **Depuración del Frontend:** Al concluir el desarrollo del frontend, se llevará a cabo su depuración, corrigiendo errores que afecten el rendimiento o la experiencia de usuario.

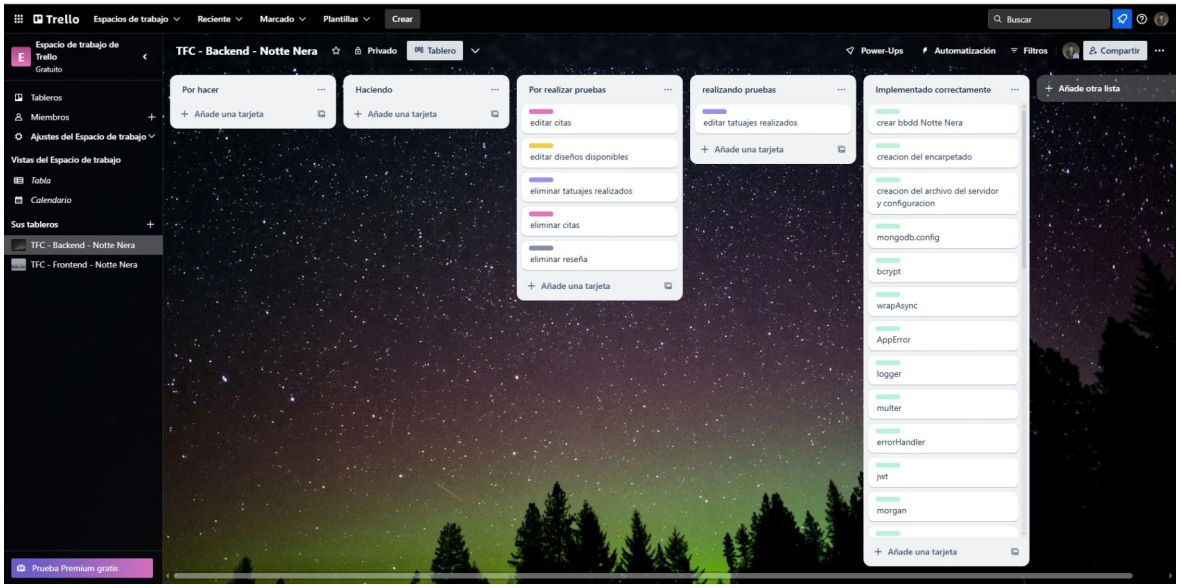
- **Validación y mejora:** Como paso final, se pedirá la opinión de usuarios externos con el fin de evaluar su experiencia con la página. Los comentarios recibidos serán fundamentales para optimizar la funcionalidad y la eficiencia del sitio web.

3.2.2 Tareas y plazos de ejecución

El desarrollo del proyecto se organizará en fases, cada una con tareas específicas y plazos bien definidos para garantizar su finalización en el tiempo estimado.

Fase 1: Planificación (Semana 1 - Semana 2)

- Definición de las vistas y estructura de la página.
- Establecimiento del diseño estético.
- Asignación de permisos según el tipo de usuario.



Fase 2: Documentación (Semana 1 - Semana 12)

- Registro de cada fase del desarrollo para mantener información actualizada.
- Elaboración de la documentación técnica.

Fase 3: Desarrollo del Backend (Semana 3 - Semana 5)

- Configuración del entorno de desarrollo.
- Implementación del servidor y sus funcionalidades principales.

- Creación de la base de datos y gestión de datos.
- Integración del renderizado del lado del servidor.

Fase 4: Depuración del Backend (Semana 6 - Semana 7)

- Ejecución de pruebas en cada funcionalidad implementada.
- Identificación y corrección de errores en el código.

Fase 5: Desarrollo del Frontend (Semana 8 - Semana 10)

- Creación de la interfaz de usuario conforme al diseño establecido.
- Conexión del frontend con el backend.
- Optimización de la experiencia de usuario y accesibilidad.

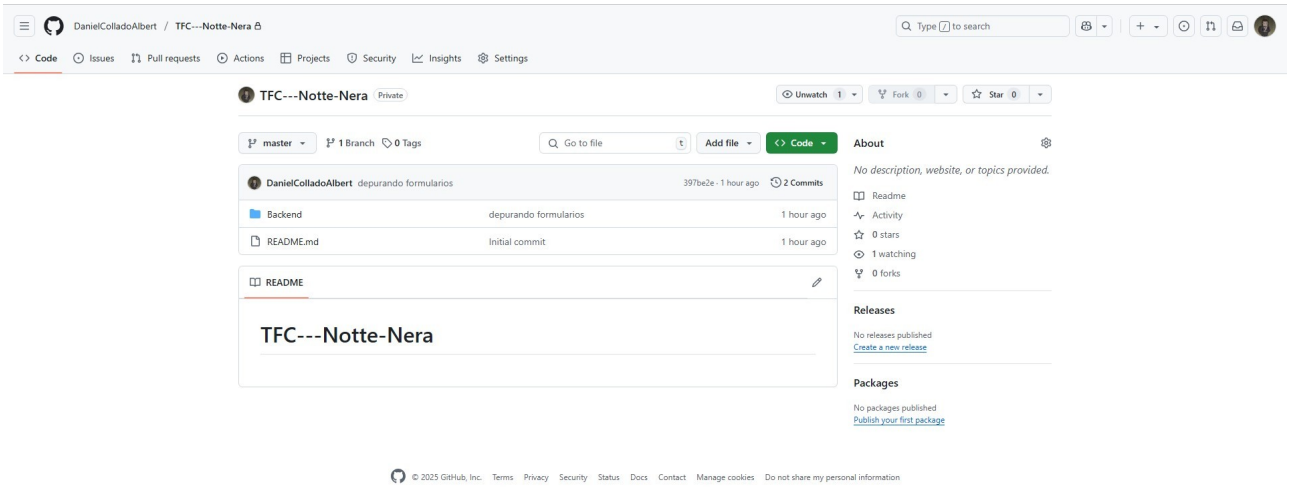
Fase 6: Depuración del Frontend (Semana 11 - Semana 12)

- Pruebas de interacción y funcionalidad.
- Corrección de errores que afecten el rendimiento o la experiencia del usuario.

Fase 7: Validación y Mejora (Semana 13)

- Recopilación de opiniones de usuarios externos.
- Implementación de mejoras basadas en el feedback recibido.
- Preparación de la versión final del proyecto.

3.2.3 Repositorio GIT



3.3 Bases de Datos

1. **Seed base**:
    a. **Propósito**: Este seed actúa como el eje central que ejecuta todos los demás seeds en el orden correcto.
    b. **Funcionalidad**:
        I) Ejecuta cada uno de los seeds individuales en secuencia.
        II) Maneja mensajes de éxito o error para cada seed ejecutado.
        III) Asegura que si un seed falla, se notifique el error.

2. **Seed usuarios**:
    a. **Propósito**: Inicializa la tabla de usuarios con datos de ejemplo.
    b. **Contenido**: Datos de usuarios ficticios necesarios para pruebas.

3. **Seed citas**:
    a. **Propósito**: Inicializa la tabla de citas con datos de ejemplo.
    b. **Contenido**: Datos de citas ficticias asociadas a usuarios existentes.

4. **Seed tatuajesRealizados**:
    a. **Propósito**: Inicializa la tabla de tatuajes realizados con datos de ejemplo.
    b. **Contenido**: Datos de tatuajes realizados asociados a usuarios y citas existentes.

5. **Seed diseñosDisponibles**:
    a. **Propósito**: Inicializa la tabla de diseños disponibles con datos de ejemplo.
    b. **Contenido**: Datos de diseños de tatuajes disponibles para ser realizados.

6. **Seed reseñas**:
    a. **Propósito**: Inicializa la tabla de reseñas con datos de ejemplo.
    b. **Contenido**: Datos de reseñas asociadas a usuarios.

Para poder ejecutar las seeds correctamente se debe abrir el terminal de Visual Studio Code y escribir el siguiente comando **"node seeds/base.seeds.js"** y al ejecutar se crearán y guardarán los diferentes datos en sus respectivas tablas.

4. Funcionamiento de la Herramienta

En este apartado se detalla el correcto funcionamiento de la aplicación web de gestión para el estudio de tatuajes. A través de capturas de pantalla y explicaciones precisas, se muestra cada funcionalidad implementada para corroborar su correcta operatividad.

4.1 Registro

En la pantalla de registro encontramos diferentes campos que el usuario debe completar para acceder a la plataforma. Los permisos de acceso y navegación estarán determinados por el tipo de usuario registrado, por defecto el rol asignado es ***"USUARIO"***:



El formulario incluye:

- Imagen de perfil
- Nombre de usuario
- Nombre de la persona
- Apellidos
- Correo electrónico

- Contraseña
- Repetir contraseña (seguridad)

Si durante el proceso de registro decidimos volver atrás, el sistema nos mostrará un mensaje de advertencia indicando que la información introducida no será guardada:



Al intentar cargar una imagen con una extensión no permitida, el sistema muestra un mensaje informativo señalando las extensiones admitidas (JPG, JPG, PNG, GIF). En caso de no subir ninguna imagen, el sistema asignará automáticamente una imagen de perfil predeterminada:



Los campos del formulario cuentan con validación en tiempo real. Si se completan incorrectamente, aparecerán mensajes de error debajo de cada campo afectado:



Si ignoramos estos mensajes de validación e intentamos finalizar el registro incumpliendo los requisitos establecidos, aparecerá un mensaje de error indicando que no se ha podido completar el registro:



4.2 Login

La pantalla de inicio de sesión presenta dos campos principales: usuario y contraseña, que deben coincidir con los datos previamente registrados:



Si alguno de los valores introducidos es incorrecto, el sistema mostrará un mensaje de error indicando que el usuario y/o contraseña son incorrectos. No se permitirá el acceso hasta que se introduzcan credenciales válidas:



Cuando las credenciales son correctas, el sistema autenticará al usuario, mostrará un mensaje de bienvenida y redirigirá automáticamente a la página principal de la aplicación:

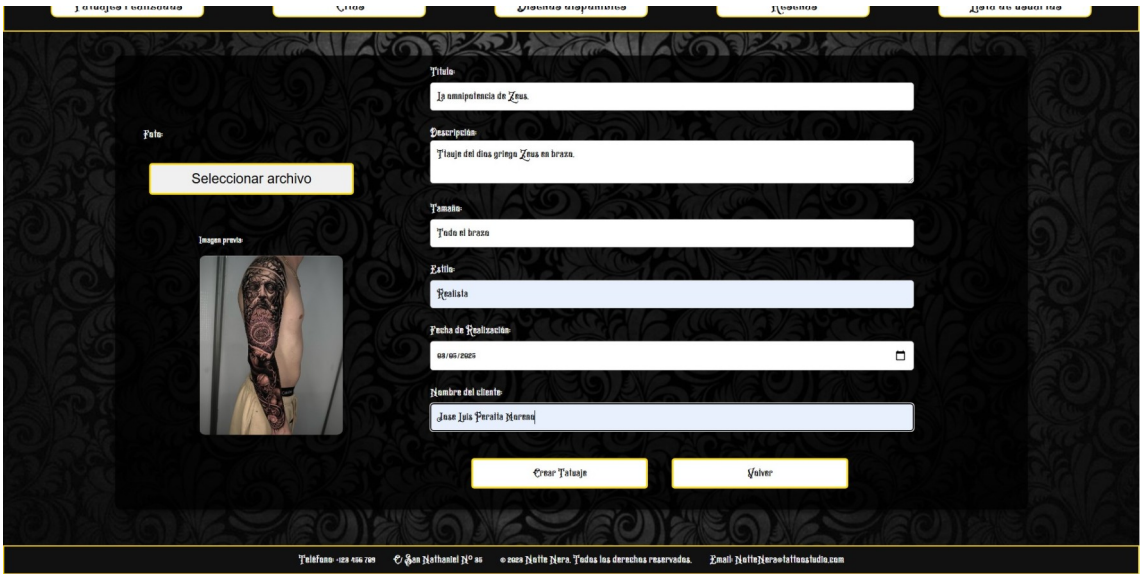


4.3 Tatuajes Realizados

Esta sección permite la gestión completa de los trabajos realizados por el artista del estudio, sirviendo como portafolio digital.

4.3.1 Crear Tatuajes Realizados

Los administradores pueden añadir nuevos trabajos al portafolio mediante un formulario específico:



El formulario incluye:

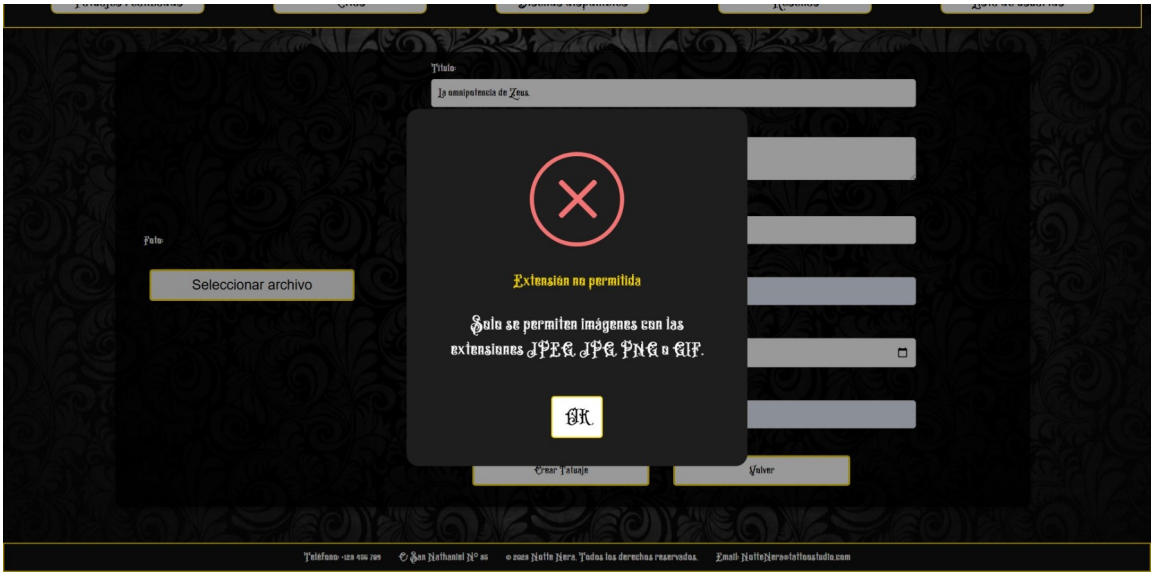
- Foto del tatuaje
- Título
- Descripción del proyecto
- Tamaño de la pieza
- Estilo artístico
- Fecha y hora de creación.
- Nombre completo del cliente

Al completar correctamente el formulario y enviar la información, se mostrará un mensaje de confirmación:

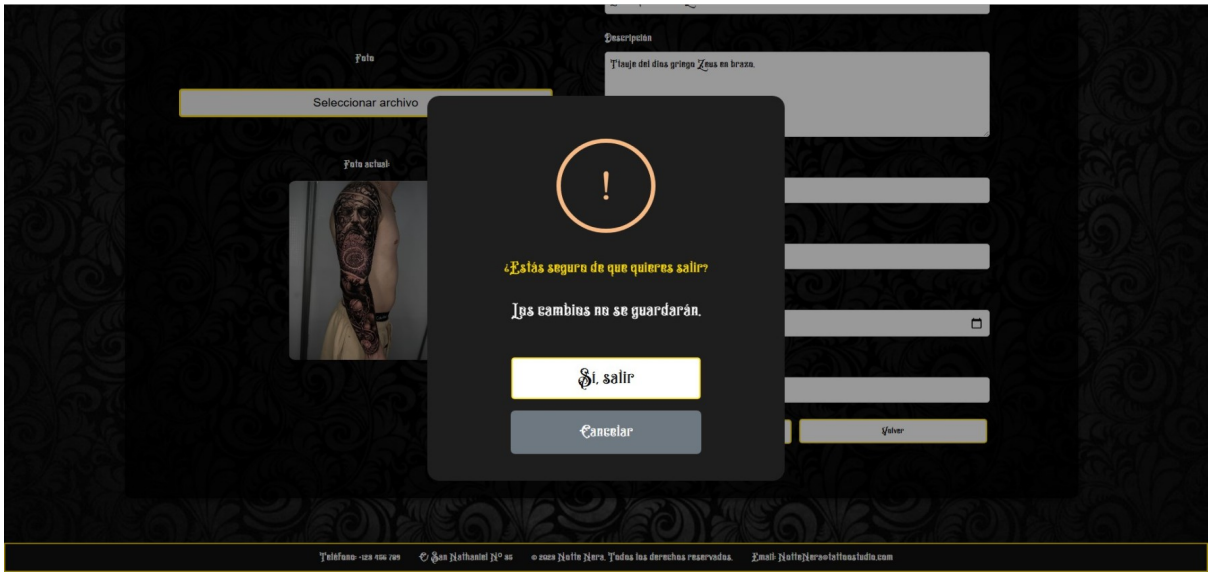
:



Por el contrario, si surge cualquier error o imprevisto el sistema nos lo hará saber con un mensaje de error:

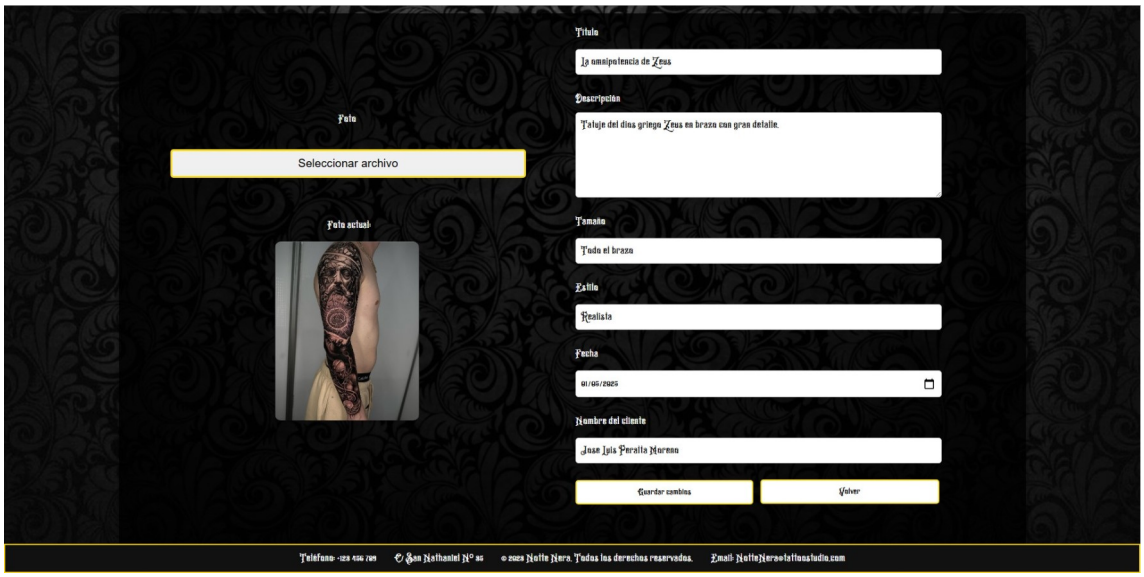


Por último, añadimos un mensaje para avisar al usuario, si este desea volver, de que los datos no se guardaran:

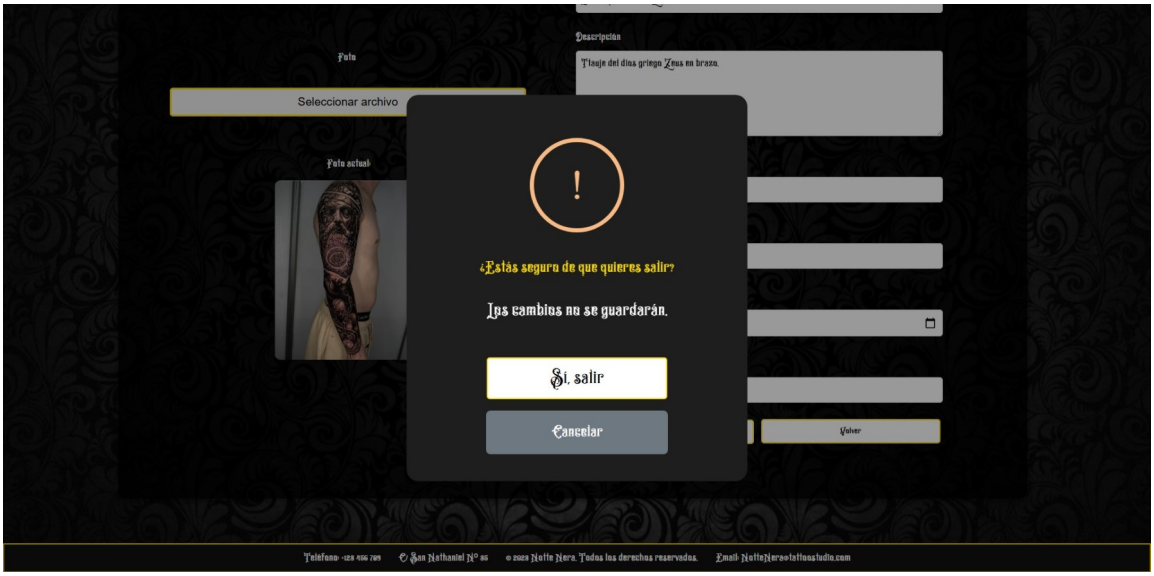


4.3.2 Editar Tatuajes Realizados

Los trabajos ya publicados pueden ser modificados por los administradores:



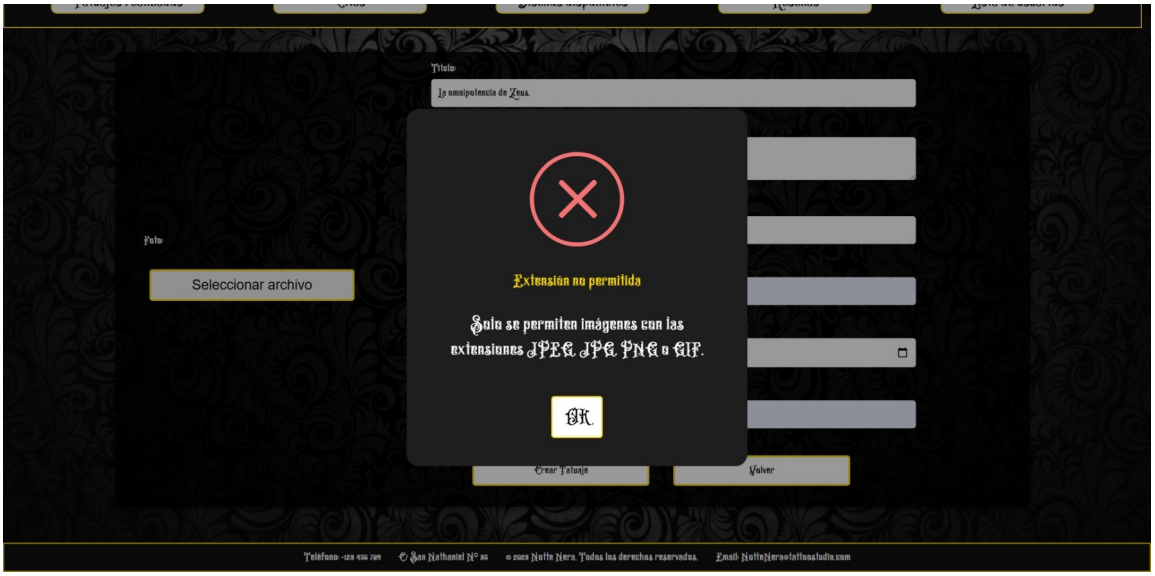
De nuevo, nos aseguramos de que, si se quiere volver no se guardaran los cambios pertinentes:



El sistema verifica que los cambios son correctos y están dentro de los estándares establecidos, y si es así, envía un mensaje de su correcta edición:



Si intentamos guardar cambios con información inválida, el sistema nos informara con un mensaje detallado:



4.3.3 Eliminar Tatuajes Realizados

La plataforma permite eliminar registros de trabajos realizados, con un sistema de confirmación para prevenir eliminaciones accidentales:



Una vez confirmada la eliminación, el sistema mostrará una notificación de éxito:

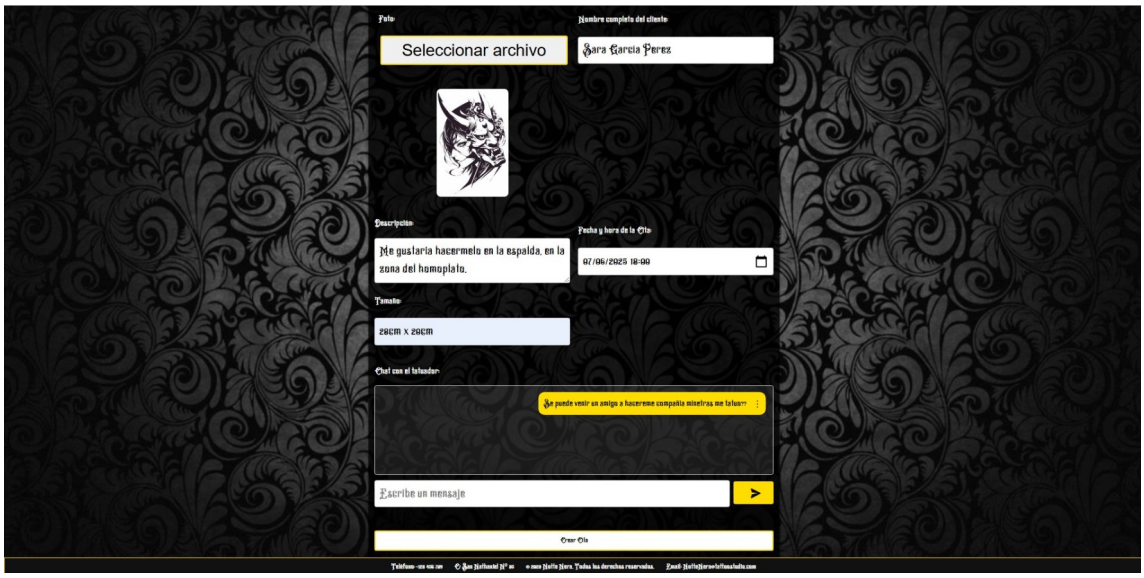


4.4 Citas

La funcionalidad de gestión de citas permite organizar eficientemente la agenda del estudio, tanto para el artista como para clientes.

4.4.1 Crear Citas

Los clientes pueden solicitar citas a través de un formulario intuitivo:



El formulario incluye:

- Foto del diseño del tatuaje
- Descripción del proyecto
- Tamaño de la pieza
- Nombre completo del cliente
- Fecha y hora
- Chat de texto con el tatuador para preguntas o reajuste de horario.

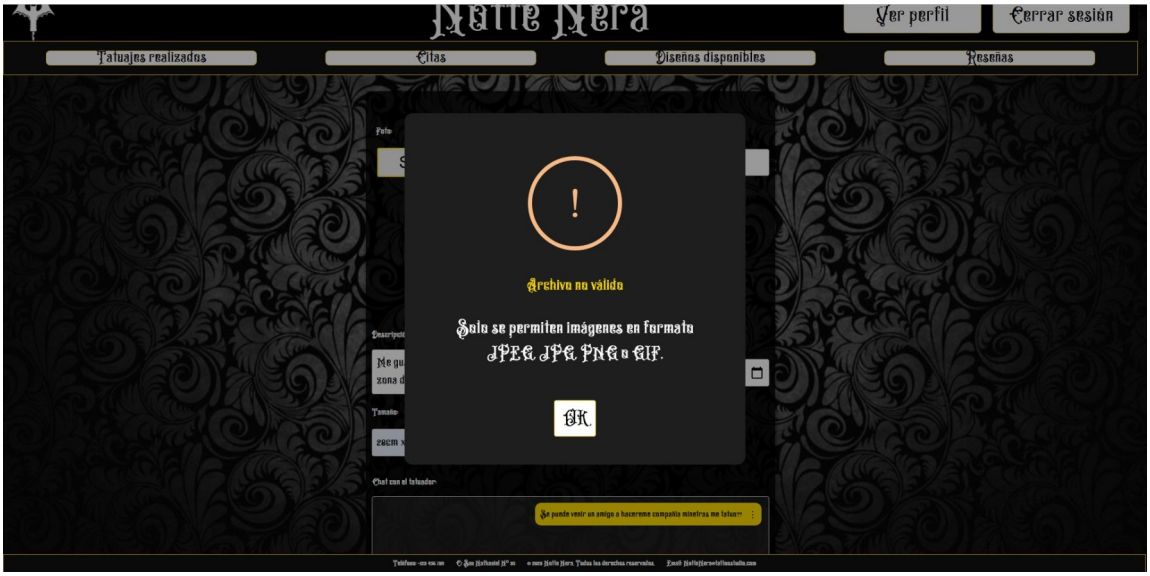
Al completar la solicitud correctamente, el sistema muestra un mensaje de confirmación y el tatuador podrá ver todas las citas en la página de inicio ordenadas de la más cercana a la más lejana, para así organizar mejor su agenda:



Por otro lado, si alguno de los campos es incorrecto se nos informara con un mensaje explicativo:

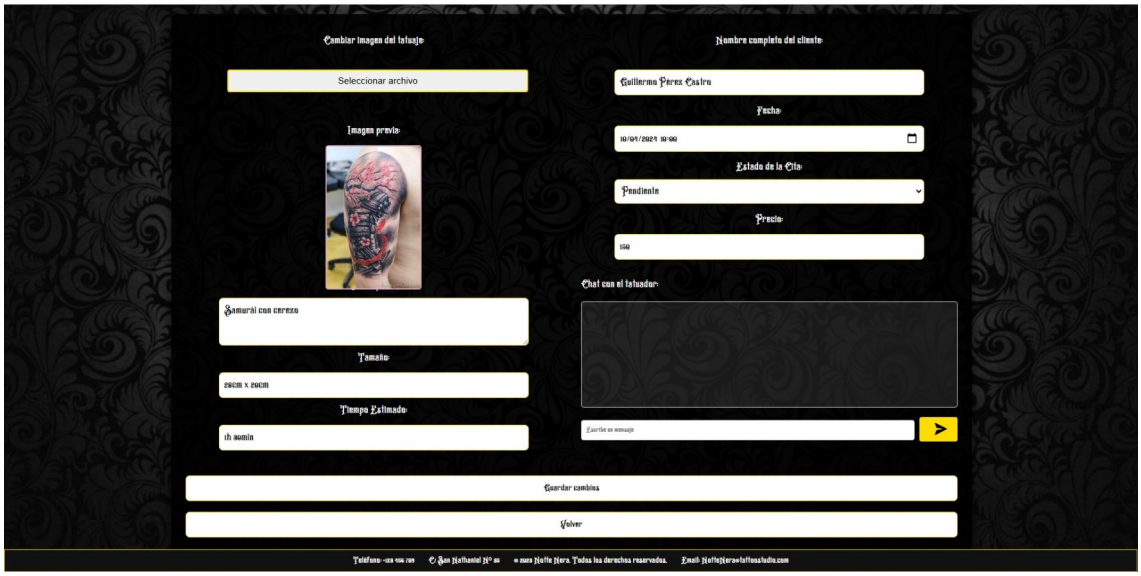


Y al intentar volver con algún cambio realizado se nos informara de que los cambios no se guardaran:



4.4.2 Editar Citas

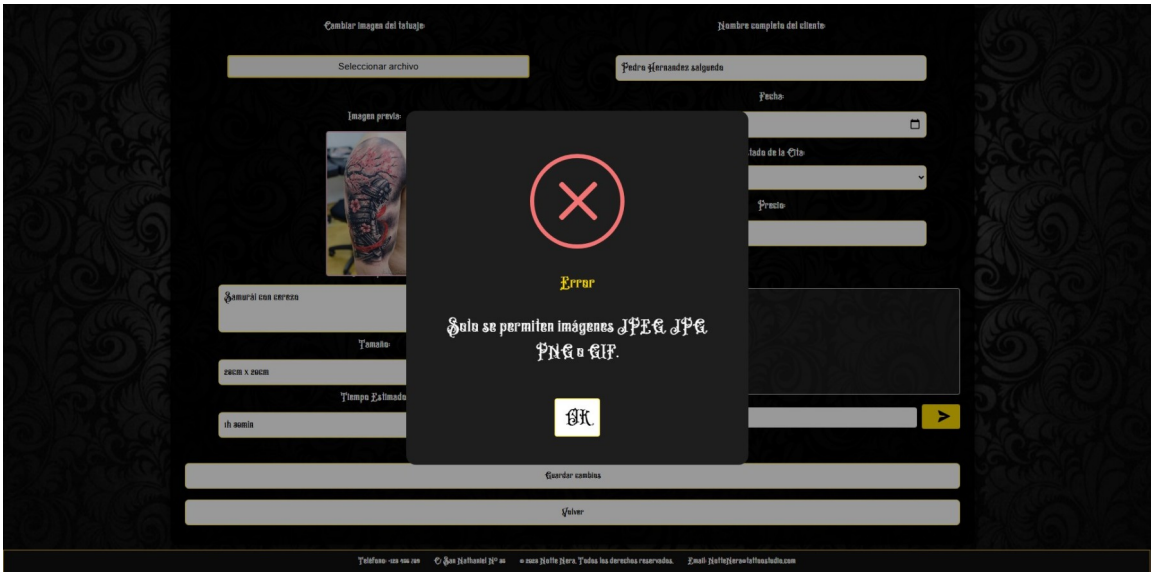
Tanto artistas como clientes pueden modificar los detalles de una cita programada:



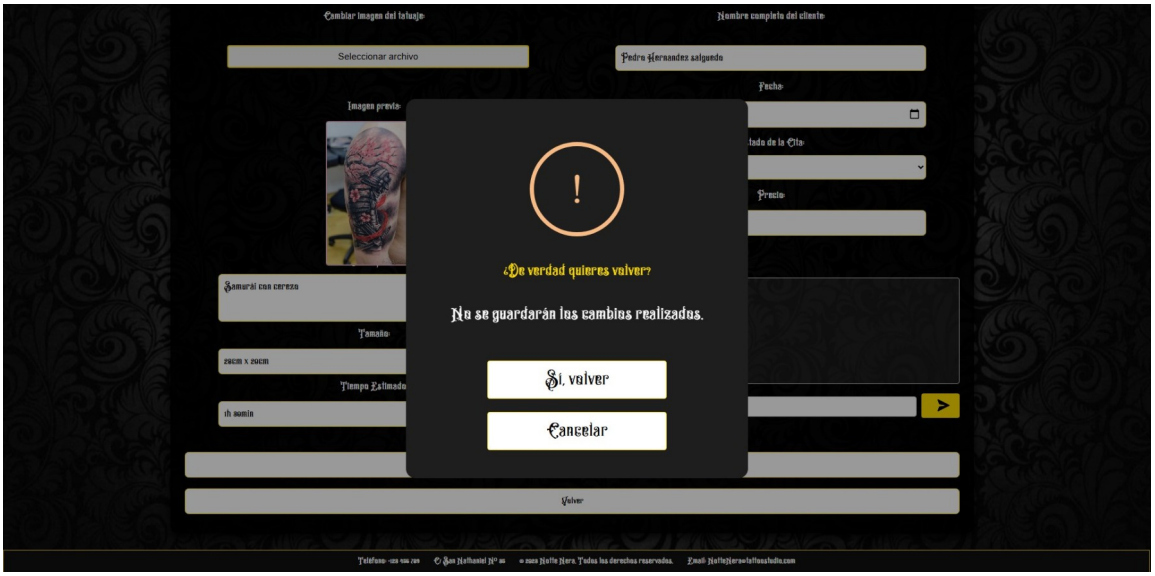
Tras realizar cambios válidos, se muestra una confirmación:



En cambio, si se ha introducido algún campo erróneamente no se guardará la cita y se nos mostrará un mensaje explicando nuestro error:



Para evitar volver atrás sin querer se ha implementado un mensaje de aviso para advertir que no se guardaran los cambios en caso de seguir adelante:



4.4.3 Eliminar Citas

Al intentar eliminar una cita se mostrará un mensaje de advertencia para evitar clics no intencionales:

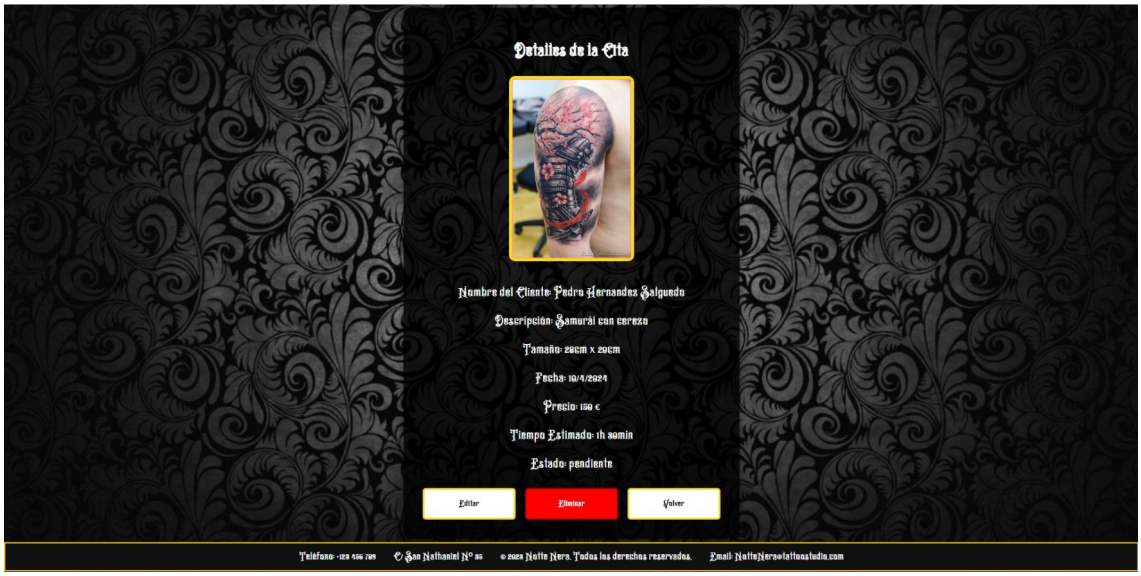


Si aceptamos la cita se eliminará exitosamente y se lo haremos saber al usuario con un mensaje explicativo:



4.4.4 Ver Detalles Citas

También podemos tener una mejor visualización de los detalles de la cita entrando en ver detalles:

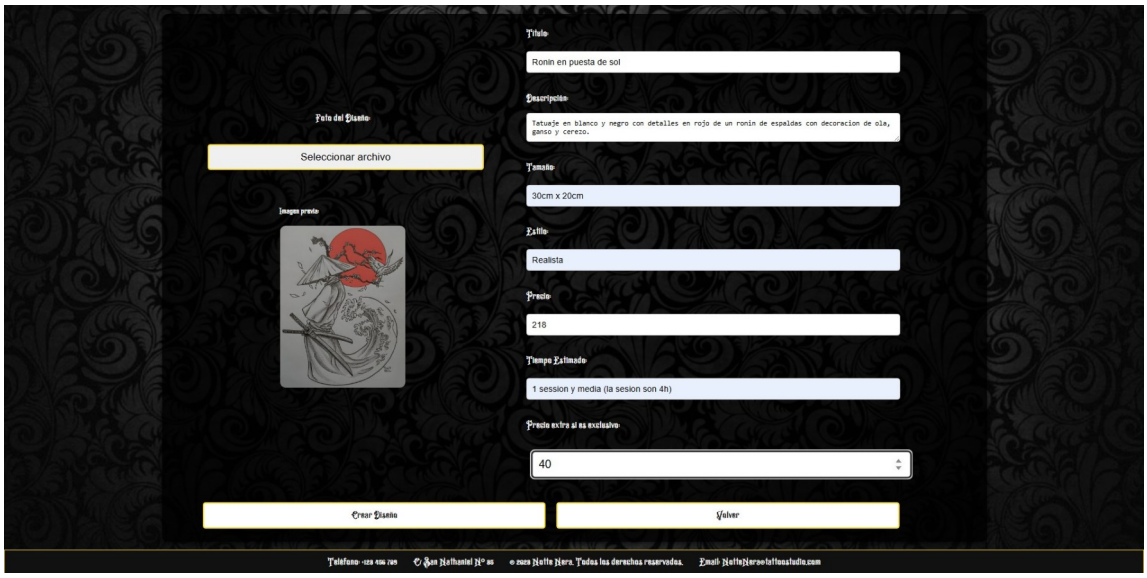


4.5 Diseños Disponibles

Esta sección gestiona el catálogo de diseños prediseñados que los clientes pueden reservar directamente.

4.5.1 Crear Diseños Disponibles

El artista puede publicar nuevos diseños disponibles para su reserva:



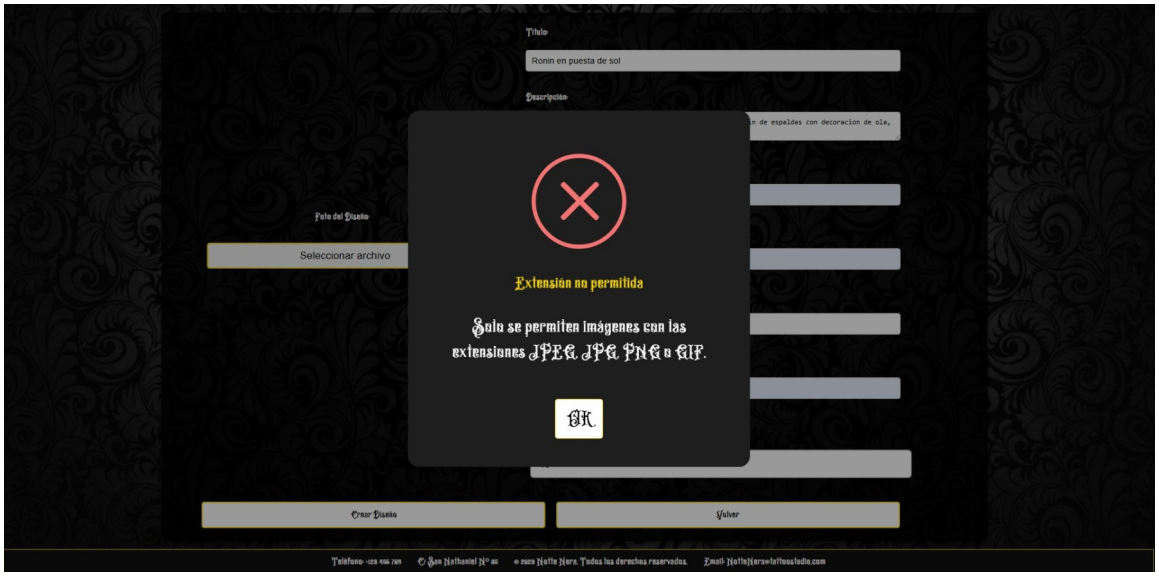
El formulario incluye:

- Imagen del diseño
- Título y descripción del diseño
- Descripción
- Tamaño recomendado
- Estilo
- Precio estimado
- Tiempo estimado de realización
- Precio extra si se quiere que sea exclusivo

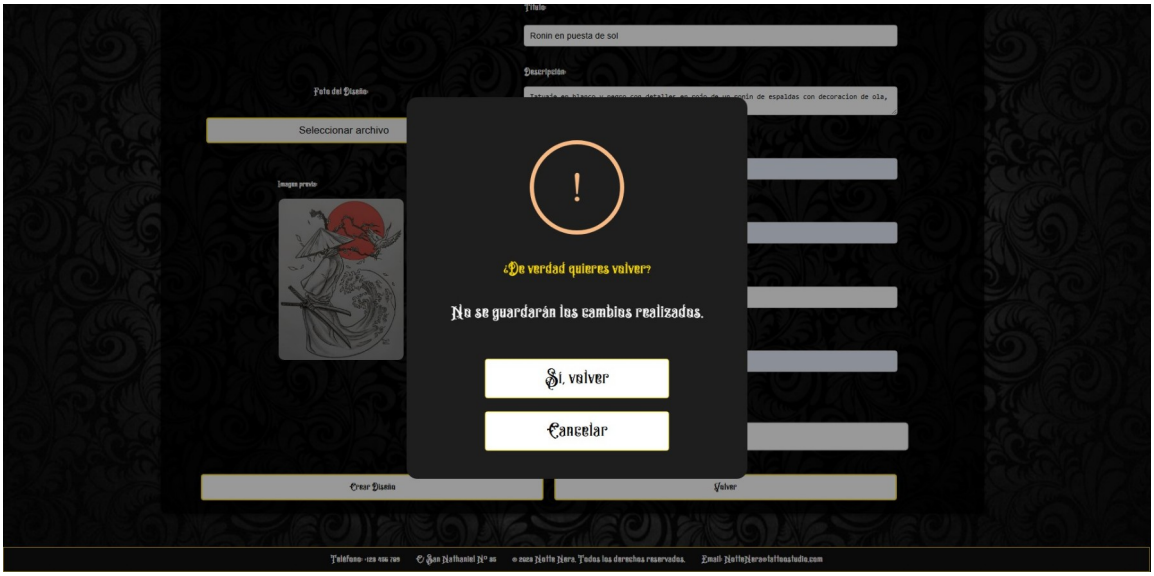
Una vez publicado, saldrá un mensaje de confirmación:



En caso de introducir algún parámetro incorrectamente se avisará de que no se ha podido guardar el diseño con un mensaje de error:

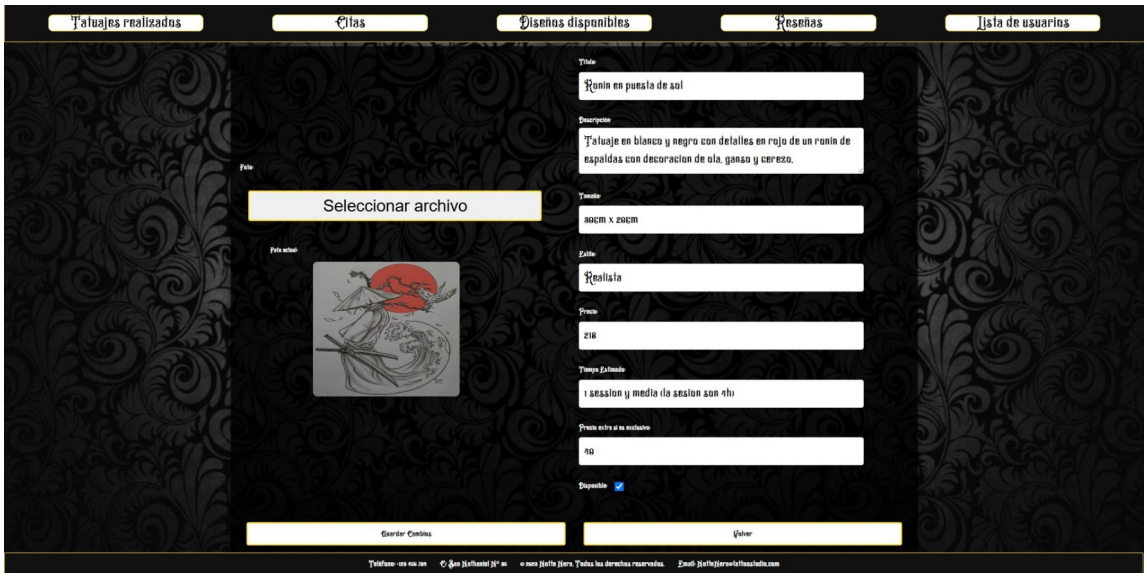


Y si se desea volver a la página anterior habiendo modificado algún valor original, saldrá un mensaje advirtiéndonos de que no se guardaran los cambios:



4.5.2 Editar Diseños Disponibles

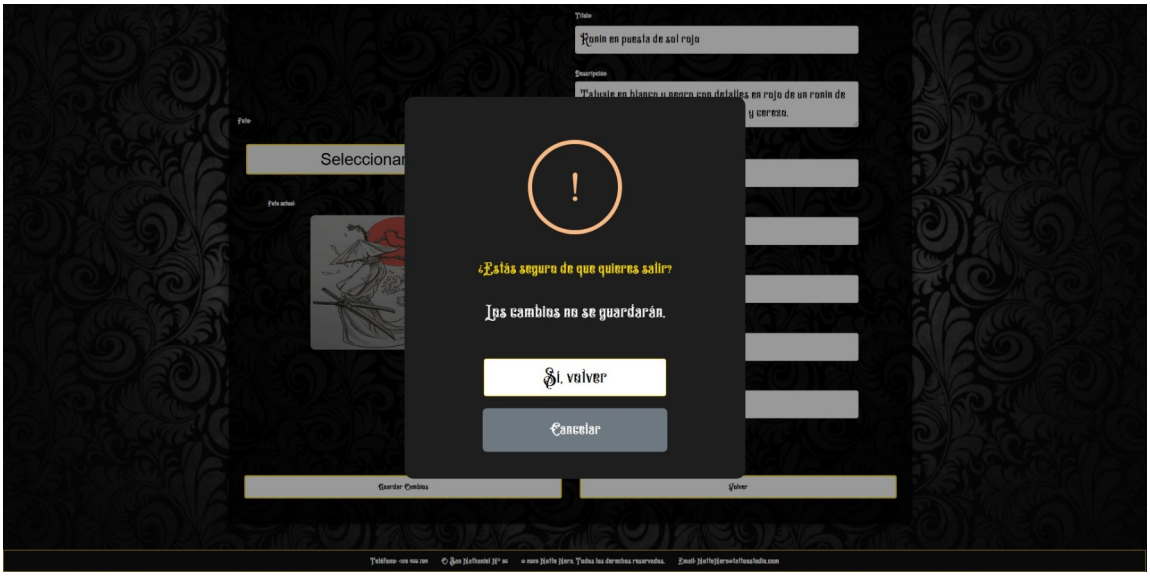
Los artistas pueden actualizar la información de sus diseños disponibles:



Los cambios realizados se reflejan instantáneamente en el catálogo:



Para evitar futuros problemas, nos aseguramos de que la extensión de la imagen es la permitida:



4.5.3 Eliminar Diseños Disponibles

Los diseños pueden ser retirados del catálogo cuando sea necesario, primero asegurando de que se quiere eliminar:

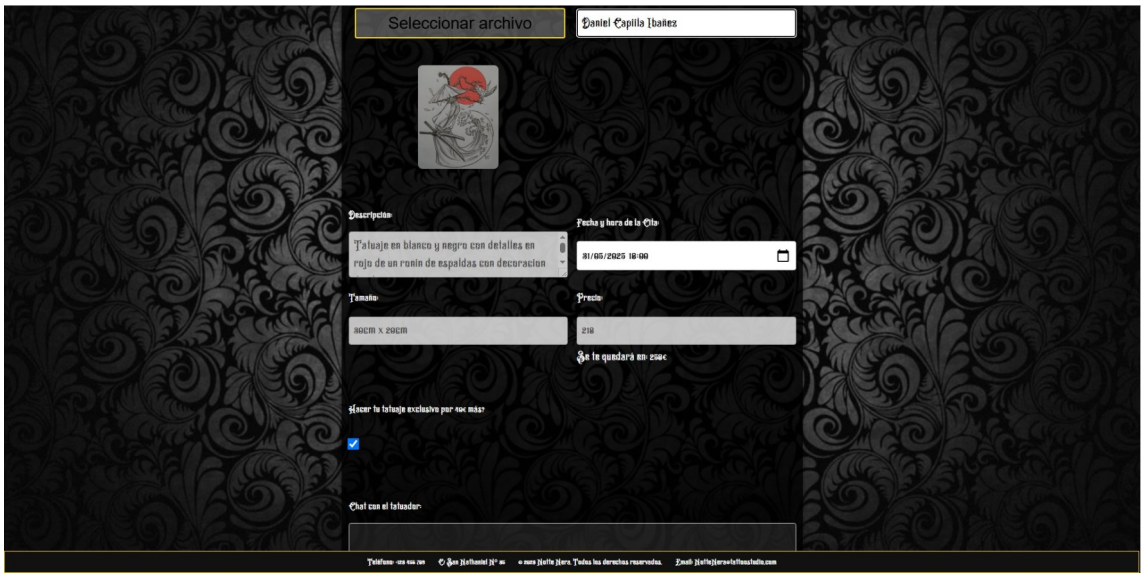


Si se sigue adelante, saldrá un segundo mensaje confirmándonos su correcta eliminación:



4.5.4 Reservar Diseños Disponibles

Los clientes pueden reservar directamente los diseños disponibles en el catálogo:



El proceso incluye:

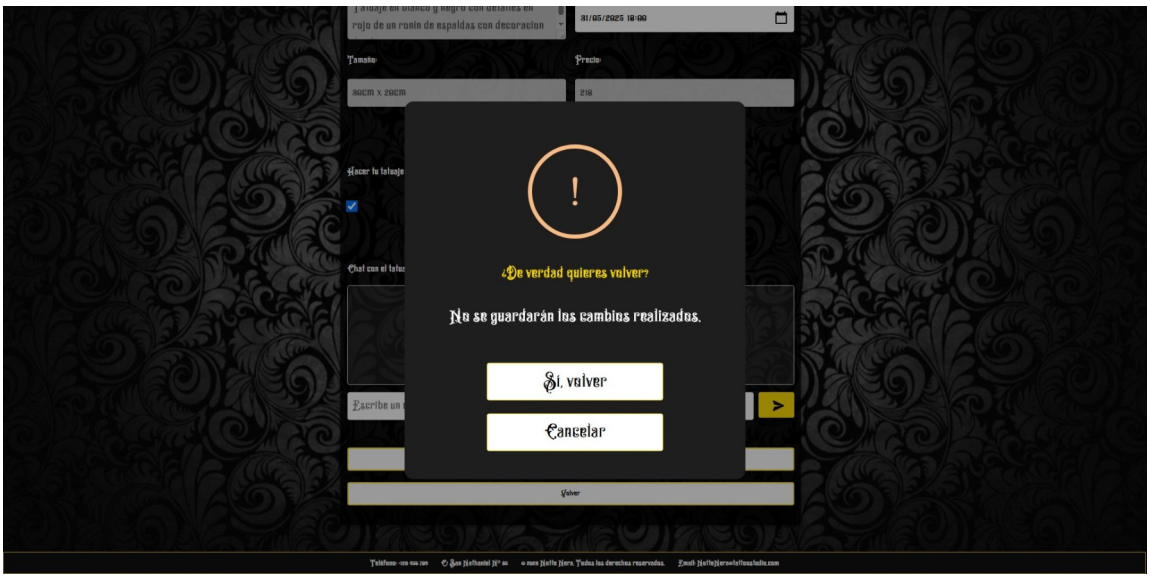
- Nombre completo del cliente
- Fecha y Hora
- Hacer exclusivo el tatuaje
- Chat de texto con el tatuador

Los demás campos están inactivos debido a que no tendría sentido que un diseño ya prestablecido puedas cambiar ciertos valores, como la foto.

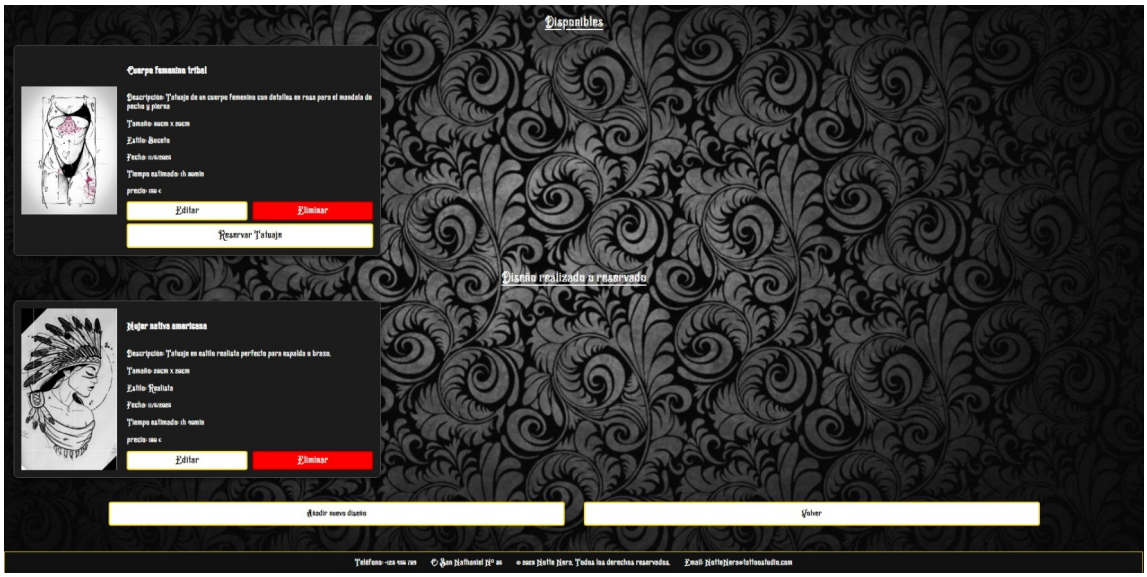
Tras completar la reserva, se genera una cita automáticamente, se envía un mensaje de confirmación y se nos redirigirá a la página de inicio de citas:



Si deseamos volver, pero hemos realizado algún cambio, se nos avisara de que estos no se guardaran:



El sistema actualiza la disponibilidad del diseño, ocultándolo del catálogo a los usuarios y para los administradores les saldrá los diseños que han sido reservados o cuales siguen disponibles:



4.6 Reseñas

El sistema de reseñas permite a los clientes valorar su experiencia con el tatuador y el estudio, creando una valiosa retroalimentación visible para futuros clientes.

4.6.1 Crear Reseñas

Los clientes pueden dejar reseñas contando su experiencia, relatando el trato, la higiene, la profesionalidad, etc.



El formulario incluye:

- Campo de texto para comentarios detallados
- Puntuación en estrellas (de 1 a 5)

Se ha previsto los posibles errores que pueda cometer el usuario y se ha tenido en cuenta para advertir de sus errores y que los corrija:



Al publicar la reseña, aparece en la sección general de reseñas tras haber sido aprobada por el administrador:



Tras la revisión manual del administrador para ver que la reseña no tenga palabras inadecuadas, sea spam..., se cambiará el estado a aprobado o cancelado según sea conveniente y las reseñas aprobadas serán las que vean por fin los usuarios.



4.6.2 Editar Reseñas

Los clientes pueden modificar sus reseñas, tanto el comentario como las estrellas, y estas actualizara su estado automáticamente a “por aprobar”, para volver a revisar la reseña:



Al guardar los cambios se nos notificara su correcta actualización con un mensaje de éxito:



4.6.3 Eliminar Reseñas

Los usuarios pueden eliminar sus propias reseñas o si son administradores pueden eliminar cualquier reseña publicada, pero antes deben aceptar un aviso para asegurarse de que realmente quieren borrar la reseña:



Al aceptar saldrá de nuevo otro mensaje con un mensaje de éxito con la eliminación:



4.6.4 Filtro y búsqueda de Reseñas

La plataforma ofrece diversas opciones para filtrar y buscar entre las reseñas publicadas:

Los usuarios pueden filtrar por:

- Rango de puntuación
- Fecha de publicación

Los resultados se actualizan en tiempo real conforme se ajustan los filtros:

Mejor valorados:



Mas antiguas:



Además, los administradores podrán filtrar por:

- **Estado**



4.6.5 Estadísticas de Reseñas

El sistema genera automáticamente estadísticas basadas en las reseñas recibidas:

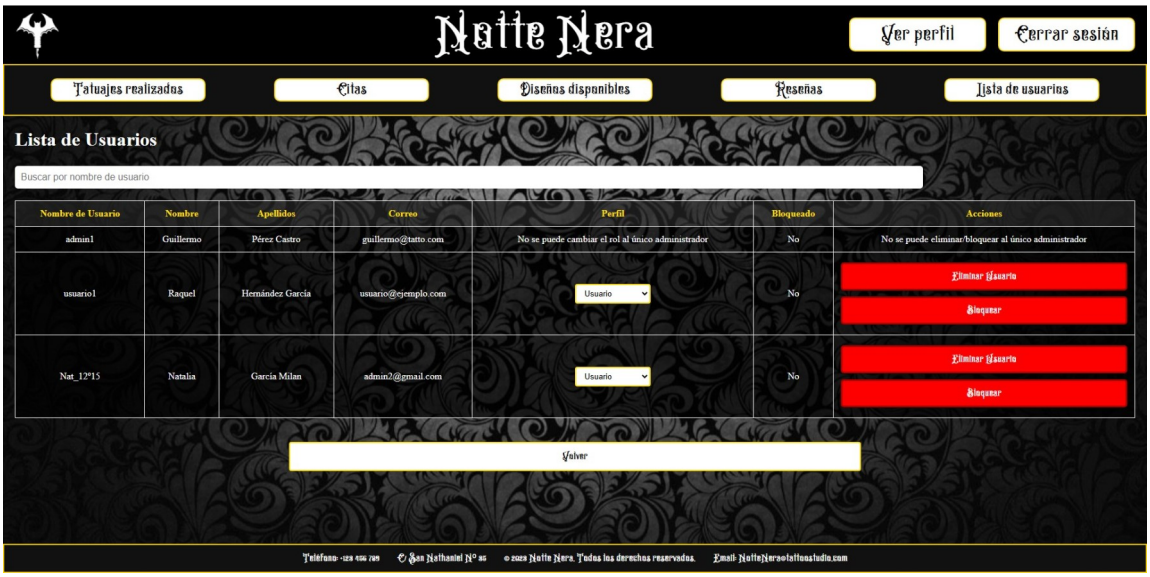


Las estadísticas incluyen:

- Total de reseñas
- Puntuación media
- Recuento de votos por cada estrella

4.7 Lista de Usuarios

Esta sección, exclusiva para administradores, permite gestionar todos los usuarios registrados en la plataforma.



4.7.1 Bloquear Usuario

Los administradores pueden suspender temporalmente el acceso de usuarios que incumplan las normas:



El proceso de bloqueo incluye:

- Campo obligatorio para el motivo

El sistema mantiene un registro detallado de los bloqueos y se lo hará saber al usuario bloqueado mediante un mensaje:

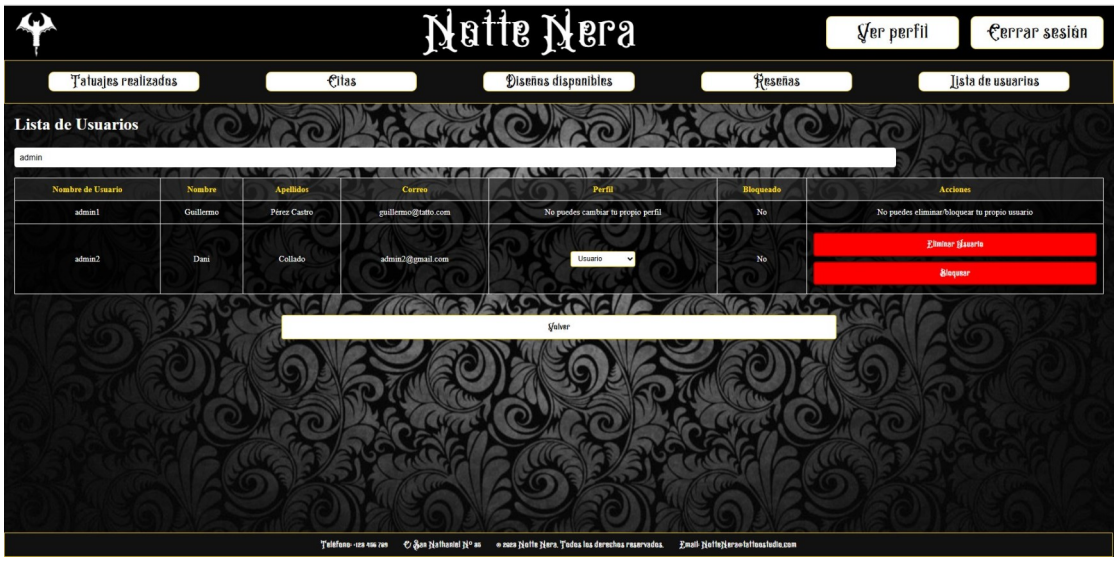


Y una pantalla de la cual no se podrá mover de ella y lo único que podrá hacer es cerrar sesión:



4.7.2 Buscar Usuario

Se dispondrá de una barra de búsqueda que permita localizar rápidamente usuarios específicos:



4.7.3 Cambiar Perfil

Los administradores pueden modificar el perfil del usuario para darle mayores privilegios sobre la página o para degradar a un administrador a usuario, al hacerlo saldrá un mensaje de aviso, para evitar futuros malentendidos:



Y tras aceptar se cambia el perfil del usuario correctamente:



Los cambios permitidos incluyen:

- Nivel de permisos (usuario, administrador)

4.7.4 Eliminar Usuarios

En casos excepcionales, los administradores pueden eliminar completamente cuentas de usuario, avisando de que se va a eliminar a un usuario con un mensaje:



Y después de confirmar su eliminación saldrá un segundo mensaje de éxito:



Cada una de estas funcionalidades ha sido implementada siguiendo los requisitos especificados y probada exhaustivamente para garantizar su correcto

funcionamiento. La interfaz de usuario ha sido diseñada priorizando la experiencia del usuario, con validaciones claras y mensajes informativos que guían el proceso en cada paso.

5. Plan de Despliegue Local

Este apartado describe de manera detallada el procedimiento necesario para desplegar y ejecutar correctamente la aplicación web completa (Frontend, Backend y Base de Datos) en un entorno de desarrollo local. El objetivo principal es asegurar que cualquier persona pueda replicar el entorno y poner en funcionamiento la aplicación sin dificultad alguna. Se trata de una guía técnica que incluye todos los pasos, configuraciones y herramientas requeridas para lograr un despliegue funcional.

5.1 Preparación del Entorno de Desarrollo

Para poder iniciar la instalación y despliegue de la aplicación, se deben cumplir los siguientes requisitos previos:

- Tener instalado Node.js y npm.
- Contar con MongoDB instalado y en ejecución en el entorno local.
- Disponer de Git para clonar el repositorio o en este caso descargarlo desde la url del OneDrive.
- Contar con una herramienta de gestión de bases de datos como MongoDB Compass para facilitar la comprobación del estado de la base de datos.
- Tener instalado un navegador web (por ejemplo, Google Chrome o Mozilla Firefox).
- Se recomienda tener instalado Vite globalmente en el sistema si se desea ejecutar el Frontend manualmente con vite (npm install -g vite).

Una vez verificados estos requisitos, se puede continuar con la instalación del proyecto.

5.2 Clonación y Configuración del Proyecto

El primer paso es clonar el repositorio desde GitHub. Para ello, se debe abrir una terminal y ejecutar el siguiente comando:

git clone https://github.com/DanielColladoAlbert/TFC---Notte-Nera.git

A continuación, se debe acceder al directorio del proyecto y proceder a instalar todas las dependencias necesarias para el funcionamiento tanto del Backend como del Frontend con el comando: **npm install**

5.3 Inicialización de la Base de Datos

Una vez instaladas las dependencias es necesario poblar la base de datos con datos iniciales.

Para ello, se ha incluido un script de seeds que primero verifica y elimina si hay datos anteriores e inserta datos de ejemplo imprescindibles para el correcto funcionamiento de la aplicación. El script se ejecuta con el siguiente comando: **node seeds/base.seeds.js**

Finalizada la ejecución, se puede comprobar desde MongoDB Compass que las colecciones han sido creadas correctamente y que los documentos han sido insertados.

5.4. Iniciar la Aplicación en Modo de Desarrollo

Para poner en marcha la aplicación, se deben levantar el servidor Backend y el cliente Frontend en paralelo.

Primero, se inicia el servidor Backend con el siguiente comando: **Nodemon**

A continuación, en una nueva terminal, se accede al directorio del Frontend y se ejecuta: **npm run dev**

Este comando arrancará la aplicación Frontend a través de Vite, en el puerto 5173. Una vez ambos servicios estén en ejecución, se puede abrir un navegador y acceder a: **http://localhost:5173** para verificar el correcto funcionamiento de la interfaz de usuario.

5.5 Desarrollo y Pruebas

Durante la fase de desarrollo, se recomienda el uso de herramientas como Insomnia para realizar pruebas sobre las rutas de la API. Gracias al uso de **Nodemon** y **Vite**, cualquier cambio en el código será reflejado automáticamente en el navegador o terminal, agilizando así el proceso de desarrollo.

Para facilitar la depuración, se pueden utilizar funciones como **console.log()** o las herramientas de depuración integradas en el entorno de desarrollo. Es importante revisar los logs tanto del servidor como del navegador para detectar posibles errores de configuración o lógica.

5.6 Validación Final

Antes de dar por finalizado el proceso de despliegue local, se deben realizar pruebas de validación para asegurar el correcto funcionamiento de todas las funcionalidades de la aplicación. Estas pruebas incluyen:

- Pruebas manuales a través de la interfaz gráfica, navegando por los distintos apartados de la aplicación, como el registro, inicio de sesión, reserva de citas, gestión de usuarios, visualización de diseños, etc.

- Pruebas de integración, asegurando que la comunicación entre el Frontend, el Backend y la Base de Datos se realiza sin errores.

5.7 Casuísticas y Recomendaciones Adicionales

- En caso de que el puerto 3016 (Backend) o 5173 (Frontend) esté en uso, se debe modificar en el archivo .env o configurar manualmente los puertos de los servidores.

- Si el comando nodemon no está disponible, se puede instalar globalmente mediante el siguiente comando: **npm install -g nodemon**

- El archivo .env no debe ser subido al repositorio en ninguna circunstancia. Debe estar listado en el archivo **.gitignore** para evitar exponer información sensible.

6. Conclusiones

El desarrollo y despliegue de esta aplicación web para la gestión de reservas en estudios de tatuajes ha sido un proyecto integral que abarca desde la planificación y diseño hasta la implementación y pruebas en un entorno de producción. A lo largo de este proceso, se han alcanzado varios objetivos clave y se han superado desafíos significativos, lo que ha permitido obtener valiosas conclusiones:

6.1 Logros Alcanzados

- **Interfaz Intuitiva y Facilidad de Uso**:
  - Se ha desarrollado una interfaz de usuario intuitiva que permite a los clientes y administradores gestionar citas, ver portafolios de artistas, y dejar reseñas de manera eficiente. La experiencia de usuario ha sido mejorada significativamente, facilitando la interacción con la plataforma.

- **Panel de Control del Administrador**:
  - El panel de control del administrador proporciona una visión completa y herramientas efectivas para gestionar los usuarios de la plataforma. Esto ha permitido una administración más eficiente y centralizada del estudio de tatuajes.

- **Sistema de Reseñas y Valoraciones**:
  - La integración de un sistema de reseñas ha permitido a los clientes dejar feedback valioso, ayudando a otros clientes a tomar decisiones informadas y proporcionando retroalimentación para mejorar los servicios ofrecidos.

- **Seguridad**:
    - Se han implementado medidas robustas de seguridad para proteger los datos de los usuarios.

6.2 Desafíos y Soluciones

- **Integración de Tecnologías**:
    - La integración de tecnologías como MongoDB, Node.js, y React ha presentado desafíos en términos de compatibilidad y rendimiento. Sin embargo, se han superado mediante una cuidadosa planificación y pruebas exhaustivas.
- **Gestión de Datos y Seeds**:
    - La gestión de datos y la ejecución de seeds para inicializar la base de datos han requerido una atención especial para asegurar la consistencia y precisión de los datos. Se ha logrado mediante la implementación de un seed principal que coordina la ejecución de otros seeds.

6.3 Futuras Mejoras

- **Ampliación de Funcionalidades**:
    - Se planea ampliar las funcionalidades de la plataforma, infomación del tatuador, información sobre ferias o exposiciones que vaya a asistir, así como un apartado de preguntas frecuentes, avisos por email a los

tatuadores cuando se haya creado o modificado una cita, entre otras mejoras.

- **Optimización Continua**:
    - Se continuará optimizando la aplicación para mejorar el rendimiento y la experiencia de usuario, basándose en el feedback de los usuarios y las métricas de rendimiento.
- **Expansión de la Plataforma**:
    - Se explorarán oportunidades para expandir la plataforma a otros mercados y regiones, permitiendo al estudio expandir su clientela y alcance.

En conclusión, este proyecto ha sido un esfuerzo exitoso para desarrollar una plataforma integral que satisface las necesidades específicas de los estudios de tatuajes. Ha proporcionado una base sólida para futuras mejoras y expansiones, y ha demostrado el valor de una planificación cuidadosa y una ejecución meticulosa en el desarrollo de aplicaciones web.

7. Bibliografía

- https://expressjs.com/en/guide/routing.html
- https://es.react.dev/learn
- https://jwt.io/introduction
- https://medium.com/@diego.coder/subida-de-archivos-con-node-js-express-y-multer-55e99219d754
- https://sweetalert2.github.io/
- https://www.youtube.com/watch?v=6u1RHUoXIPI
- https://www.youtube.com/watch?v=usajPd4YMZs